Spenso

Typed tensors, symbolic structures, and executable tensor networks

---

1 Overview

Spenso represents and evaluates tensors with abstract indices. It separates a tensor's structure—representations, slots, dimensions, names, and index order—from its stored data. Dense and sparse storage, symbolic and parametric values, automatic index matching, and network execution build on that separation.

> **Two questions, two layers**
> Tensor structure answers "which indices and representations does this object carry?" Tensor data answers "which values occupy that structure?" Contraction first matches compatible slots, then combines data. Keep these phases distinct when diagnosing a shape, duality, or contraction error.

1.1 Choose a task

- To install Symbolica and construct a typed tensor with the bundled Python module, use the Python guide.
- To verify slot duality with one numerical Rust contraction, follow the Rust guide, then continue with the first-contraction tutorial and the exact `Contract` reference.
- To assemble and execute several tensor/scalar nodes, use the tensor-network guide with the `Network` reference.
- To construct tensors or execute a network from Python, follow the Python tensor-workflow guide with the exact `TensorNetwork` reference.
- To parse symbolic tensor functions or apply Dirac/color identities, review the feature and interface guide, then continue with Idenso for representation-aware rewrites.

1.2 From tensors to networks

The core package supports three related levels of work:

- tensor structures describe slots and abstract indices;
- dense, sparse, numeric, symbolic, and parametric tensors attach data to those structures;
- tensor networks represent a collection of tensors and choose an execution or contraction strategy.

Networks can be assembled directly. With the `symbolica` feature, they can also be inferred from Symbolica expressions whose functions carry recognizable tensor structure. Parsing is not mere string conversion: representation initialization, index variance, names, and the tensor library determine the inferred network.

```
[dependencies]
spenso = "0.6"
```

Enable `symbolica` only when Symbolica-backed parsing or symbolic operations are needed. It pulls in the Symbolica integration and corresponding Linnet support. The default core remains useful for typed tensors and contractions without that layer.

1.3 Companion packages

> **Component ownership**
> `spenso` owns the tensor model and execution engine. `spenso-macros` owns the `SimpleRepresentation` derive

used to declare custom index representations. `spenso-hep-lib` owns concrete high-energy-physics tensor-library data such as gamma matrices and projectors. The `spynso3` adapter owns the Python community module. Their versions and feature gates are independent, so select and upgrade each component explicitly.

Spenso uses Linnet for the underlying network graph. Idenso is the symbolic-identity layer for Spenso-formatted Symbolica expressions. GammaLoop consumes these components inside a broader collider workflow.

The interface guide, native Rustdoc, and generated Python reference list the public modules, traits, feature gates, and import paths.

Contributors changing structures, contraction, parsing, storage, or network scheduling should also read the source-audited Spenso implementation architecture.

2 Build your first Spenso tensor

Spenso is available as a native Rust library and as a community module bundled with Symbolica's Python package. Both interfaces preserve representation, variance, and abstract-index structure alongside tensor data.

Python
Start here for notebooks and exploratory calculations. The published Symbolica wheel includes the native Spenso community module and needs no GammaLoop checkout.

Use Spenso from Python ⟶

Rust
Start here when a Rust application owns tensor storage, contractions, or network execution. The published `spenso` crate exposes the complete native API.

Use Spenso from Rust ⟶

Choose Python for the shortest installation and Rust for the most complete library surface. The interface guide maps both routes to their generated reference.

3 Using Spenso from Python

The shortest path into Spenso is the community module bundled with Symbolica `2.2.0`. This example creates a typed two-dimensional tensor and checks its diagonal entries; it needs no GammaLoop checkout.

3.1 Install and verify the module

Create an isolated environment, install the version used by this manual, and verify the community import:

```
python -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install "symbolica==2.2.0"
python -c "import symbolica.community.spenso as spenso; print(spenso.__name__)"
```

There is no separate `spenso` Python wheel. The installed `symbolica` distribution owns the native `symbolica.community.spenso` module. Symbolica's installation and license terms apply; its free restricted mode is enough for this single-process example.

3.2 Construct a dense tensor

Save this as `spenso_quickstart.py`:

```
from symbolica.community.spenso import Representation, Tensor, TensorIndices

rep = Representation.euc(2)
matrix = Tensor.dense(
    TensorIndices(rep("i"), rep("j")),
    [1.0, 0.0, 0.0, 1.0],
)

assert matrix[0, 0] == 1.0
assert matrix[1, 1] == 1.0
matrix.to_sparse()
print(matrix)
```

Run `python spenso_quickstart.py`. `TensorIndices` gives the data a representation-aware structure, while the four row-major values provide its concrete storage. Converting to sparse storage changes only that storage; it does not change the slots or re-index the tensor.

> **Structure comes before storage**
> Equal dimensions alone do not make two tensor slots compatible. Their representations, variance, and abstract indices remain part of the value. Diagnose those properties before changing dense or sparse storage.

Continue with the Python tensor-workflow guide, or use the Rust guide to perform a checked native contraction.

4 Using Spenso from Rust

This example constructs two dense tensors, contracts one pair of dual indices, and checks one output component. It uses a two-dimensional Euclidean representation as a compact software exercise rather than a Lorentz-space physics example.

4.1 Create a Rust project

Use Rust 1.85 or newer:

```
cargo new spenso-quickstart
cd spenso-quickstart
cargo add spenso@0.6.0
```

Replace `src/main.rs` with:

```
use spenso::{
    contraction::Contract,
    structure::{
        Canonicalized, OrderedStructure,
        representation::{Euclidean, LibraryRep, RepName},
        slot::{DualSlotTo, IsAbstractSlot},
    },
    tensors::data::{DenseTensor, GetTensorData, SetTensorData},
};

fn main() {
    let rep = Euclidean {};
    let left_free = rep.new_slot(2, 0).to_lib();
    let right_free = rep.new_slot(2, 2).to_lib();
    let shared = rep.new_slot(2, 10).to_lib();

    let left_structure: OrderedStructure<LibraryRep> =
        Canonicalized::from_iter([left_free, shared]).into_canonical();
```

```
    let right_structure: OrderedStructure<LibraryRep> =
        Canonicalized::from_iter([right_free, shared.dual()]).into_canonical();

    let mut left = DenseTensor::<i32, _>::zero(left_structure);
    let mut right = DenseTensor::<i32, _>::zero(right_structure);
    left.set(&[0, 1], 2).unwrap();
    right.set(&[0, 1], 5).unwrap();

    let result = left.contract(&right).unwrap();
    assert_eq!(*result.get_ref([0, 0]).unwrap(), 10);
    println!("result structure: {}", result.structure);
    println!("result data: {:?}", result.data);
}
```

Run `cargo run`. Success means the assertion passes and the printed result contains `10` at the component selected by the two remaining free indices. Spenso matches index identity and duality, not merely equal dimensions.

> **The docs run this example**
> The documentation test compiles and executes this program against the current workspace. The contraction itself needs neither Symbolica nor Spenso's optional `symbolica` feature.

Continue with the first tensor workflow for a more deliberate explanation of structure and storage, then use the tensor-network guide when more than two tensors should be planned and executed together.

5 Tutorial

This tutorial performs one concrete tensor contraction in Rust. The example makes the tensor structure, dual index matching, storage, and numerical result visible—the same layers that a larger Spenso network coordinates automatically. It uses a two-dimensional Euclidean representation as a small linear-algebra example; it is not a Lorentz-space physics example.

For the shortest installation and first-run path, begin with Using Spenso from Rust. Then return here for a closer look at the index structure and contraction result.

5.1 Prerequisites

Use Rust 1.85 or newer (Spenso uses Rust 2024 edition), then create a binary project:

```
cargo new spenso-first-contraction
cd spenso-first-contraction
cargo add spenso@0.6
```

This first contraction uses dense integer tensors and needs neither Symbolica nor the `symbolica` feature.

5.2 Contract one shared index

Replace `src/main.rs` with:

```
use spenso::{
    contraction::Contract,
    structure::{
        Canonicalized, OrderedStructure,
        representation::{Euclidean, LibraryRep, RepName},
        slot::{DualSlotTo, IsAbstractSlot},
    },
    tensors::data::{DenseTensor, GetTensorData, SetTensorData},
```

```
};

fn main() {
    let rep = Euclidean {};
    let left_free = rep.new_slot(2, 0).to_lib();
    let right_free = rep.new_slot(2, 2).to_lib();
    let shared = rep.new_slot(2, 10).to_lib();

    let left_structure: OrderedStructure<LibraryRep> =
        Canonicalized::from_iter([left_free, shared]).into_canonical();
    let right_structure: OrderedStructure<LibraryRep> =
        Canonicalized::from_iter([right_free, shared.dual()]).into_canonical();

    let mut left = DenseTensor::<i32, _>::zero(left_structure);
    let mut right = DenseTensor::<i32, _>::zero(right_structure);
    left.set(&[0, 1], 2).unwrap();
    right.set(&[0, 1], 5).unwrap();

    let result = left.contract(&right).unwrap();
    assert_eq!(*result.get_ref([0, 0]).unwrap(), 10);
    println!("result structure: {}", result.structure);
    println!("result data: {:?}", result.data);
}
```

Run `cargo run`. Success means the assertion passes and the printed result contains `10` at the component selected by the two remaining free indices. Spenso matched `shared` with `shared.dual()`, summed that axis, and preserved `left_free` and `right_free` in the output structure. The contraction follows index identity and duality, not merely equal dimensions.

> **Verification scope and cost**
> The docs harness compiles and runs this Rust program; it syntax-checks the setup commands without creating a project or using the network. Success is the asserted component value `10` with two free output indices. The contraction is tiny; a clean external Cargo build can take minutes, while the program itself should finish in well under a second.

> **Structure is part of the value**
> The data vectors alone do not say which axes may contract. Preserve the `OrderedStructure` with the tensor, and inspect the resulting structure before interpreting a flat component offset. A dimension match with unrelated abstract indices produces an exterior product, not the contraction shown above.

5.3 From two tensors to a network

Pairwise `Contract` is the clearest first success. For a larger expression, place tensors in a `Network`, inspect its graph, choose or benchmark a contraction strategy, and execute it. Use `DataTensor` when individual nodes may be dense or sparse. Enable `symbolica` only when the network must mix concrete data with Symbolica-backed parametric tensors.

5.4 Troubleshooting and next steps

- If `contract` leaves both shared-looking axes free, print their slots and check that one is the dual of the other; dimensions alone are insufficient.
- If `set` rejects a component, compare the coordinate count and each coordinate with the structure's rank and dimensions.
- If a sparse contraction reports an empty input, distinguish an intentionally zero tensor from a tensor whose nonzero entries were never populated.

- Continue with the tensor-structures and networks manual before selecting an automatic contraction strategy.

6 Tensor data, contraction, and network execution

Spenso keeps tensor structure and tensor data separate so the same structural operations work with dense, sparse, symbolic, or parametric storage. A structure supplies ordered slots, representations, dimensions, and abstract indices. Data supplies values and a storage-specific access strategy. Contraction first establishes structural compatibility and only then chooses a data operation.

6.1 Dense, sparse, and parametric data

Dense storage is appropriate when most components are populated and a predictable contiguous layout matters. Sparse storage is appropriate when zero structure dominates, but contraction cost depends on index overlap and lookup behavior rather than only the number of stored values. Parametric and symbolic tensors defer numeric substitution; keep their parameter environment with the tensor when serializing or moving a calculation between stages.

Conversions must preserve the structure's slot order. A tensor with the same dimensions but a different representation or variance is not interchangeable until an explicit structural operation establishes that relationship.

6.2 Contraction and planning

Pairwise contraction matches compatible dual slots, determines the output structure, and then contracts the selected data backends. A tensor network generalizes this to many nodes connected through abstract indices. The current public API selects a contraction strategy while executing the network; it does not expose a separately stored, reusable plan object.

The cheapest-looking local pair is not always the cheapest complete order. Intermediate tensor dimensions, sparsity, symbolic expression growth, and reusable subexpressions can dominate. Record the strategy and step bound with performance results. A structural change can lead the same strategy to choose a different order.

> **Strategy selection is not a cached plan**
> Replacing values without changing slots preserves the contraction problem, but the public API still runs the selected strategy during execution. Adding a tensor, changing a representation, or rewiring an index changes that problem and requires a fresh execution decision.

6.3 Execute a small tensor/scalar network

This self-contained network computes `2 a + 3 b` for two equally structured tensors. The dummy libraries make the execution boundary explicit: every tensor is already concrete and any unresolved function key is an error.

```
use spenso::{
    network::{
        ExecutionResult, Network, Sequential, SmallestDegree,
        library::{DummyKey, DummyLibrary, DummyLibraryTensor, panicing::ErroringLibrary},
        store::NetworkStore,
    },
    structure::{
        OrderedStructure,
        representation::{Euclidean, RepName},
    },
    tensors::data::DenseTensor,
};

fn main() {
    type Tensor = DenseTensor<f64, OrderedStructure<Euclidean>>;
    type Store = NetworkStore<Tensor, f64>;
```

```
    type Net = Network<Store, DummyKey, DummyKey>;
    type LibraryTensor = DummyLibraryTensor<Tensor>;

    let structure = OrderedStructure::new(vec![Euclidean {}.new_slot(2,
1)]).into_canonical();
    let a = DenseTensor::from_storage_data(vec![1.0, 2.0], structure.clone()).unwrap();
    let b = DenseTensor::from_storage_data(vec![3.0, 4.0], structure).unwrap();
    let tensors = DummyLibrary::new();
    let functions = ErroringLibrary::new();

    let mut network =
        Net::from_scalar(2.0) * Net::from_tensor(a) + Net::from_scalar(3.0) *
Net::from_tensor(b);
    network
        .execute::<Sequential, SmallestDegree, LibraryTensor, _, _>(&tensors, &functions)
        .unwrap();

    let ExecutionResult::Val(result) = network.result_tensor::<LibraryTensor,
_>(&tensors).unwrap()
    else {
        panic!("expected a concrete tensor result");
    };
    assert_eq!(result.data, vec![11.0, 16.0]);
}
```

The output invariant is the component vector `[11.0, 16.0]`; the original rank-one structure must also remain attached to the result. The documentation harness compiles the complete program; run it to execute the assertion in a provisioned Symbolica environment. See the exact `Network`, `DenseTensor`, and pairwise contraction Rustdoc.

> **Interpret execution failures by layer**
> `from_storage_data` errors are rank/dimension or storage-length mismatches. An unresolved function-key error means the network contains a symbolic library node despite the concrete-only setup. A different value with the same shape points to scalar placement or node arithmetic; a different shape points to changed slots or unintended contraction, which requires replanning.

6.4 Symbolica shadowing

With `symbolica`, Spenso recognizes registered tensor functions inside Symbolica expressions and builds a network whose nodes shadow those subexpressions. Initialization of representations and the tensor library must happen before parsing. Unknown functions remain symbolic rather than silently acquiring tensor semantics.

Shadowing is useful for extracting a contraction problem while retaining a reversible link to the source expression. Keep the replacement map until the computed result has been substituted back. For rewrite identities and index cooking, use Idenso Spenso itself owns storage, contraction, and execution.

The implementation handoff from symbolic syntax into network nodes is traced in the Spenso parsing flow. The Symbolica syntax and rewrite guide records the contributor-facing matching and replacement conventions.

6.5 Custom representations and the HEP library

`SimpleRepresentation` derives the repetitive representation plumbing for a user enum. The derive input still defines domain semantics: duality, dimension, slot compatibility, and any Symbolica tags must agree. The resulting representations implement the ordinary Rust traits used by the rest of Spenso; the `spenso-macros` API reference lists the supported derive attributes.

`spenso-hep-lib` registers concrete high-energy-physics structures such as gamma matrices and projectors. Treat that library as shared domain data. GammaLoop uses the same definitions.

Graph and tensor ownership
Linnet owns connectivity, cuts, cycles, and graph mutation. Spenso owns which connected slots contract and how a plan is executed. Keeping that boundary explicit prevents a graph rewrite from being mistaken for a tensor-algebra identity.

7 Python tensor workflows

Spenso's Python interface is a native adapter over the same tensor structures, libraries, and network executor as the Rust crates. Use the generated reference for exact signatures and this guide for the object boundaries and execution sequence that those signatures do not explain.

7.1 Availability and version boundary

A Symbolica community module
Import Spenso as `symbolica.community.spenso`; there is no standalone `spenso` wheel. The published Symbolica wheel bundles this community module. Its Symbolica version determines which Spenso API it contains. A source checkout or generated `.pyi` file does not add the native module to an existing Python environment.

Install the current assembly and check the environment before building a workflow:

```
python -m pip install --upgrade symbolica
python -c "import symbolica.community.spenso as spenso; print(spenso.__name__)"
```

Source embedders can build a custom community-module assembly. Record the Symbolica assembly version with reproducible results; it is a more useful Python compatibility fact than the version of an unrelated local Rust checkout.

Spenso, Idenso, and Symbolica core share one native library, one Symbolica kernel, and one Python `Expression` type. The community assembly registers `SpensoModule` as `symbolica.community.spenso_native`; its public wrapper imports the native exports and calls the host-provided `initialize_module`. That initializer must remain in the export list. Do not link Spynso into `gammaloop._gammaloop` or distribute it as a second native extension.

GammaLoop owns the Spynso source and bundled Tydenso `render.typ` and `notation.typ` assets; Symbolica Community owns the wheel. Their native dependencies must resolve one Symbolica source revision. The `gammaloop[typst-display]` extra adds only the optional renderer, not another Spynso binary.

7.1.1 Pyodide builds

The adapter's default `native` feature includes native arithmetic and C++ evaluator compilation. For Pyodide, the community assembly must depend on `spynso3` with `default-features = false` and forward its `wasm` feature to `spynso3/wasm`. Every other Symbolica consumer in that assembly must also disable native defaults and select its portable arithmetic features; Cargo combines features across dependencies.

`TensorEvaluator` remains available in this configuration. `TensorEvaluator.compile` and `CompiledTensorEvaluator` require the `native` feature and are absent in Pyodide.

With Python 3.14, `pyodide-build==0.39.0`, `maturin==1.15.0`, and the `314.0.7` cross-build environment installed, run the following from the community assembly's directory. Its manifest must define the `release-small` profile and forward `wasm` as described above; the GammaLoop workspace root builds a different Python package.

```
export RUSTUP_TOOLCHAIN="$(pyodide config get rust_toolchain)"
rustup target add --toolchain "$RUSTUP_TOOLCHAIN" wasm32-unknown-emscripten
pyodide build . --outdir dist --no-isolation \
```

```
  -C maturin.build-args="--locked --profile release-small \
  --no-default-features --features wasm \
  -- -C link-arg=-sEXPORTED_FUNCTIONS=_PyInit_core"
```

In this repository, `just check-spynso-wasm` checks the adapter for Emscripten and rejects native arithmetic or workspace-hack dependencies in its portable graph.

7.2 Choose the right object

- `Representation` and `Slot` define dimensions, duality, and abstract indices.
- `TensorExpression` carries a symbolic expression and its ordered tensor interface. A `Slot` supplies an explicit index; a `Representation` leaves a port open for later indexing.
- `TensorPattern` and `PortPattern` build rewrite patterns without requiring a concrete interface.
- `Tensor` owns dense or sparse data with a `TensorExpression` as its exact structure. Register a named tensor in a `TensorLibrary` for reuse by symbolic networks.
- `TensorNetwork` owns an expression graph. `ExecutionMode` selects the rewrite strategy: one smallest-degree rewrite per step, scalar work only, or the general smallest-degree strategy. Use `n_steps`, not the mode, to bound how many execution steps run.
- `TensorEvaluator` substitutes repeated numerical parameter batches into a symbolic tensor; its compiled counterpart owns the generated native evaluator.

The generated `Representation`, `Tensor`, and `TensorNetwork` entries are the exact versioned contracts. Do not infer index compatibility from two equal dimensions: names, representations, and duality remain part of the value.

7.3 Construct and inspect concrete data

This complete source creates a named rank-two tensor, verifies one component, and converts its storage in place. The documentation harness compiles the Python source without importing the native module.

```python
from symbolica.community.spenso import Representation, Tensor, TensorName

rep = Representation.euc(2)
i = rep("i")
j = rep("j")
structure = TensorName("A")(i, j)
matrix = Tensor.dense(structure, [1.0, 0.0, 0.0, 1.0])

assert len(matrix) == 4
assert matrix[0, 0] == 1.0
matrix.to_sparse()
assert matrix[1, 1] == 1.0
```

`Tensor.dense` requires row-major data whose length is the product of the structure dimensions. `Tensor.sparse` instead needs the element type and starts empty. `to_dense()` and `to_sparse()` mutate the storage representation; they do not change slots or re-index the tensor. See the dense constructor and conversion contract.

> **Diagnose structure before storage**
> A constructor failure usually means the data length and dimensions disagree. An unexpected contraction or exterior product is instead an index/duality problem. Print the structure and slots before changing dense/sparse storage, because a storage conversion cannot repair a structural mismatch.

7.4 Typed tensor factories and patterns

Calling a user-defined `TensorName` places scalar key arguments before structural ports. Predefined tensors instead have typed factories on `TensorExpression`: `g`, `flat`, `gamma`, `gamma5`, `projm`, `projp`, `sigma`, `f`, and `t`. Factory dimensions select representations; they are not scalar arguments and do not add fields to tensor-library keys.

```
from symbolica.community.spenso import _, TensorExpression

gamma = TensorExpression.gamma(4)
gamma_ijmu = gamma("i", "j", "mu")
line = gamma(_, _, "mu") * gamma(_, _, "nu")
dirac_trace = line.trace()

generator = TensorExpression.t(8, 3)
T_aij = generator("a", "i", "j")
```

Gamma's public argument order is its stored interface: bispinor-in, bispinor-out, then Minkowski. `_` (also exported as `AUTO`) leaves a local port unresolved; it is not a shared Einstein index. Calling a partially indexed expression assigns only its remaining open ports. Raw predefined `TensorName` accessors expose heads for inspection and matching, not concrete construction.

Patterns retain ordinary Symbolica wildcard behavior, including wildcard dimensions:

```
from symbolica import S
from symbolica.community.spenso import TensorPattern

D_, i_, j_, mu_ = S("D_", "i_", "j_", "mu_")
gamma_pattern = TensorPattern.gamma(D_, i_, j_, mu_)
```

`TensorPattern` shortcuts follow the same index order as concrete factories. General patterns place scalar `args` before structural `ports`; `PortPattern.exact`, `.any`, `.self_dual`, and `.dualizable` select fixed or constrained representation heads. Wildcard-head names are strings ending in one underscore; dimensions, indices, and scalar arguments are numbers or Symbolica expressions. Patterns can be passed directly to Symbolica replacement operations.

7.5 Register data and execute a network

A symbolic network resolves named tensors through a library. Register the data first, construct the expression with the same name and compatible slots, then execute and request the result:

```
from symbolica.community.spenso import (
    ExecutionMode,
    Representation,
    Tensor,
    TensorLibrary,
    TensorName,
    TensorNetwork,
)

rep = Representation.euc(2)
A = TensorName("A")
structure = A(rep, rep)
library = TensorLibrary()
library.register(Tensor.dense(structure, [1.0, 0.0, 0.0, 1.0]))

network = TensorNetwork(A(rep("i"), rep("j")), library=library)
network.execute(library=library, mode=ExecutionMode.All)
result = network.result_tensor(library=library)
assert len(result) == 4
```

`ExecutionMode.Single` selects the smallest-degree single-rewrite strategy, but execution still continues while work remains unless it is bounded; use `n_steps=1` to inspect exactly one step. `Scalar` processes scalar work while retaining tensor structure, and `All` attempts the complete available execution. A successful `execute()` does not

make `result_scalar()` appropriate for a tensor result; choose `result_tensor()` or `result_scalar()` according to the remaining structure. Exact behavior is linked from `execute`, `result_tensor`, and `ExecutionMode`.

> **Interpret a network failure by ownership**
> A missing-library error means a symbolic tensor name has no registered data. A result-kind error means the graph still has tensor structure when a scalar was requested, or vice versa. A changed network structure invalidates assumptions about contraction order; replacing only values with the same registered structure does not.

7.6 Mathematical display

`TensorExpression`, `Tensor`, and `TensorNetwork` share semantic display methods. `DisplaySettings` controls the ports, Schoonschip, and call layouts, dimensions, parentheses, commas, symbol scripts, and index/factor spacing. Positional calls such as `to_typst(True)` and `formatted(True)` still request dimensions.

```
from symbolica.community.spenso import DisplaySettings, TensorExpression

trace = TensorExpression.gamma5(4).trace()
source = trace.to_typst(
    settings=DisplaySettings(show_dimensions=True, parentheses=False)
)
```

`to_typst` and `format_tensor` emit source using the ports layout. Schoonschip, call, and custom-spacing settings require Tydenso's Typst notation layer; use HTML, SVG, or notebook display for those settings. Source-only methods reject unsupported settings rather than silently ignoring them.

Install the optional compiler to render HTML and SVG:

```
pip install 'gammaloop[typst-display]'
```

```
from symbolica.community.spenso import DisplaySettings, TensorExpression

trace = TensorExpression.gamma5(4).trace()
compact = DisplaySettings.schoonschip()
html = trace.to_html(settings=compact)
svg = trace.to_svg(settings=compact)
rich = trace.formatted(settings=compact)
```

Python uses the bundled Typst render/notation assets directly, without calling the Tydenso Wasm plugin. Explicit `to_html` and `to_svg` calls raise an install-guidance `ImportError` when the compiler is absent. Notebook `_repr_html_` and `formatted()` fall back to existing LaTeX or text. `TensorNetwork.__str__` remains Graphviz DOT; `to_dot()` makes that intention explicit. These display methods do not replace Symbolica's inherited `to_latex` API.

Idenso transformations still return ordinary Symbolica expressions. Module-level `spenso.formatted(expression)` or `spenso.as_tensor(expression)` provides tensor-aware display. HTML, SVG, and rich-display functions accept `notation_source` as a trusted, complete replacement for the bundled `notation.typ`, not a style fragment. Typst executes that source; it must implement the expected notation interface and must not come from untrusted input. Display customization stays outside Atom payloads; portable representation and math-label declarations continue to travel with the expressions.

7.7 Repeated symbolic evaluation

Use `Tensor.evaluator()` when the tensor structure stays fixed and only symbolic parameters change across batches. Supply constants, custom functions, and parameters explicitly. Call `evaluate()` for real inputs and `evaluate_complex()` when coefficients or inputs are complex. `compile()` creates source and a shared library, so

treat its filenames, compiler, architecture, and optimization level as reproducibility inputs rather than incidental arguments.

Control Symbolica-backed Rayon work with `SymbolicParallelism`: `Serial` keeps work on the calling thread, `Auto` checks license capability and applies workload heuristics, and `Parallel` forces Rayon without that safety choice. Configure the policy before benchmarking; otherwise a threading-policy change can be mistaken for an algorithmic improvement.

For exact parameters and defaults, use the evaluator reference and symbolic parallelism reference. The implementation starts in the Spenso Python adapter.

8 Rust, macros, and Python APIs

8.1 Core Rust package

The `spenso` crate organizes its public surface into `structure`, `tensors`, `contraction`, `network`, `iterators`, and `algebra`. The important abstractions form a progression:

- `TensorStructure` and related traits describe slots, names, and contraction compatibility;
- `DenseTensor`, `SparseTensor`, and the heterogeneous tensor enums own storage;
- contraction traits perform pairwise or multi-tensor operations;
- network stores and libraries bind symbolic tensor names to concrete data and execute a graph.

Trait implementations and generic constraints determine which combinations of structure and data support an operation. Consult the Rust orientation to choose the relevant crate, then use its revision-specific Rustdoc when a method is unavailable for a particular tensor type. Symbolica integration and symbolic parallelism controls require the `symbolica` Cargo feature.

8.2 Proc macros and HEP data

`spenso-macros` is a separate proc-macro crate. Its `SimpleRepresentation` derive generates the representation and duality boilerplate used by Spenso index types. A declaration supplies a symbolic name and chooses either `self_dual` or a `dual_name`:

```
use spenso_macros::SimpleRepresentation;
use spenso::structure::representation::RepName;

#[derive(SimpleRepresentation)]
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Default)]
#[representation(name = "flavor", dual_name = "AntiFlavor")]
struct Flavor {}
```

The `SimpleRepresentation` Rustdoc lists the derive's helper attributes and allowed targets. Macro expansion happens at compile time and produces ordinary Rust implementations.

`spenso-hep-lib` supplies domain data and tensor-library construction for high-energy physics. It is intentionally separate from the generic core. Users who need gamma matrices or physics projectors add that package; generic Spenso users do not inherit those conventions implicitly.

8.3 Python community module

An adapter, not a Spenso wheel
Python users import `symbolica.community.spenso`. The implementation is the `spynso3` Rust adapter and is distributed through the Symbolica community-module mechanism. Enabling Spenso's Rust `python` feature only enables conversion interoperability; it does not create a standalone importable `spenso` Python distribution.

Install the published Symbolica assembly with `python -m pip install --upgrade symbolica`, then verify `python -c "import symbolica.community.spenso"`. Module availability follows the Symbolica assembly version; a local Spenso source checkout does not add the module to an installed wheel. Source embedders must add `spynso3` to the external symbolica-community assembly and invoke its `SymbolicaCommunityModule` registration while building that extension; building the Rust crate alone does not inject the module into another Symbolica wheel.

```
from symbolica.community.spenso import (
    Representation,
    Tensor,
    TensorName,
)

rep = Representation.euc(2)
structure = TensorName("I")(rep("i"), rep("j"))
identity = Tensor.dense(structure, [1.0, 0.0, 0.0, 1.0])
```

The Python tensor-workflow guide connects construction, libraries, network execution, evaluators, and symbolic-parallelism policy. Use it with the structured Python reference, whose exact signatures can differ from the generic Rust API because `spynso3` provides Python-specific conversions and defaults.

Source starting points are the core crate, the derive crate, the HEP library, and the Python adapter.

9 Version history

> **History coverage**
> This version of Spenso is 0.6.0, but the published changelog currently ends at 0.5.6. Release notes for 0.6.0 are not yet available, so the history below is incomplete for that version. `spenso-macros`, `spenso-hep-lib`, and `spynso3` have independent versions and changelogs.

9.1 Available histories

- Spenso core versions
- Spenso macros versions
- Spenso HEP library versions
- Spynso3 Python-adapter versions

When upgrading, inspect the history for tensor structure, contraction, network parsing, Symbolica integration, and Python API or signature changes. Also check each companion component used by the application: a core Spenso release does not imply a release of the macro, HEP, or Python adapter package.

9.2 Upgrade checklist

Compare tensor structure and contraction behavior, network parsing, Symbolica integration, feature flags, and Python signatures for every component used by the application.

9.3 Reproducibility record

For reproducible upgrades, record the versions of Spenso and every companion component used by the application. Do not assume that changes between 0.5.6 and 0.6.0 are fully represented in the available notes.

10 Spenso versions

This page renders the canonical Spenso changelog source as part of the Spenso website, navigation, and search.

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

11.1 Unreleased

11.2 0.5.6 - 2026-01-08

11.2.1 Fixed

- fix `python_stub_gen` feature
- fix metric initialization and update to symbolica 1.2

11.2.2 Other

- update linnet

11.3 0.5.5 - 2025-12-15

11.3.1 Fixed

- fix canonization in presence of duals
- fix sum normalization in network parsing
- fix warnings
- fix overflowing slot order
- fix `add_assign` with sparse tensors

11.3.2 Other

- Update symbolica to 1.1.0
- update linnet
- add depth to parser, to stop at first mul
- remove more prints
- remove prints
- update linnet
- add readmes
- formatting and add ref for parametric
- update to 0.17 `pyo3_stub_gen`
- update linnet and make classattr to classmethods
- Release spenso-macros 0.3
- update to symbolica 1.0
- update sympolica to 0.20
- Properly precontract scalars
- Add back pyo3-stub-gen-derive
- Updated spenso with minor canonical string update for symbolica f2eb0c1ddcc61c342f2ff9616c2d129141d433dc
- Refactor coefficient conversions to support generic float types
- update to current dev symbolica and add pattern for pslash + m conjugate
- update to linnet 0.13.1
- working canonize (up to functions) and dirac adjoint
- spenso canonize buggy
- `robust_conj` but with lots of `gamma_0s`
- first try conj
- update python api
- working pow and fun execution
- working pow execution with wrapping
- support for executing pow and functions on tensors
- implement function library trait
- add function generic key

11.4 0.5.3 - 2025-08-18

11.4.1 Other

- update linnet to crates
- all tests pass
- Fix Multiple Heads Bug
- implement ExportNumber for spenso::Complex
- to Atom for ExpandedIndex
- remove printouts
- update linnet
- update linnet
- update linnet
- add infinite loop error test
- update parse problem and add spenso:: to all symbols
- remove expand

11.5 0.5.2 - 2025-07-15

11.5.1 Other

- remove printout

11.6 0.5.1 - 2025-07-15

11.6.1 Fixed

- fix gamma algebra≈

11.6.2 Other

- allow arbitrary arguments for to dots

12 Spenso macros versions

The procedural macros are versioned independently from Spenso. Their canonical version history remains the spenso-macros changelog source. This rendered copy integrates it with the manual, navigation, search, and product PDF.

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

13.1 Unreleased

13.2 0.3.2 - 2026-01-08

13.2.1 Fixed

- fix metric initialization and update to symbolica 1.2

13.2.2 Other

- update linnet

13.3 0.3.1 - 2025-12-15

13.3.1 Other

- Update symbolica to 1.1.0
- update linnet
- update linnet
- add readmes
- update linnet and make classattr to classmethods

13.4 0.2.1 - 2025-08-18

13.4.1 Other

- update linnet to crates
- update linnet
- update linnet
- update linnet

14 Spenso HEP library versions

The HEP helper library is versioned independently from Spenso. Its canonical version history remains the spenso-hep-lib changelog source. This rendered copy integrates it with the manual, navigation, search, and product PDF.

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

15.1 Unreleased

15.2 0.1.7 - 2026-01-14

15.2.1 Other

- update versions

15.3 0.1.6 - 2026-01-08

15.3.1 Fixed

- fix `python_stub_gen` feature
- fix metric initialization and update to symbolica 1.2

15.4 0.1.5 - 2025-12-15

15.4.1 Fixed

- fix warnings
- fix `add_assign` with sparse tensors

15.4.2 Other

- Update symbolica to 1.1.0
- update linnet
- add readmes

- Release spenso-macros 0.3
- update to symbolica 1.0
- update sympolica to 0.20
- Properly precontract scalars
- Refactor coefficient conversions to support generic float types
- working pow and fun execution
- working pow execution with wrapping
- add function generic key

15.5 0.1.3 - 2025-08-18

15.5.1 Other

- all tests pass
- update linnet

15.6 0.1.2 - 2025-07-15

15.6.1 Fixed

- fix gamma algebra again

15.6.2 Other

- update versions

15.7 0.1.1 - 2025-07-15

15.7.1 Fixed

- fix gamma algebra≈

15.7.2 Other

- update release action
- allow arbitrary arguments for to dots

16 Spynso3 versions

The Python adapter is versioned independently from Spenso. Its canonical version history remains the spynso3 changelog source. This rendered copy integrates it with the manual, navigation, search, and product PDF.

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

17.1 Unreleased

17.2 0.1.2 - 2026-01-14

17.2.1 Other

- update versions

17.3 0.1.1 - 2025-12-15

17.3.1 Fixed

- fix sum normalization in network parsing
- fix warnings
- fix `add_assign` with sparse tensors
- fix projp
- fix param docs and iter return type
- fix `pyo3_stubs`

17.3.2 Other

- Update implementation to pyo3-stub-gen 0.17
- Update symbolica to 1.1.0
- update linnet
- add readmes
- update to 0.17 `pyo3_stub_gen`
- update linnet and make classattr to classmethods
- move initialize to api
- Release spenso-macros 0.3
- update to symbolica 1.0
- update sympolica to 0.20
- Properly precontract scalars
- Add back pyo3-stub-gen-derive
- update to current dev symbolica and add pattern for pslash + m conjugate
- first try conj
- update python api
- add function generic key
- again
- return any because python is dumb
- update `__iter__` method
- use numpy doc format
- add overloaded pyi stub gen

Python API reference

The supported Python packages available in this version are listed below. The HTML manual provides a separate page for every public class, function, and constant; this printable edition keeps a compact inventory. Rust APIs use canonical Rustdoc in the HTML manual.

`spynso3`

Exports registered by spynso3.

`BroadcastFunction`

A unary Symbolica function whose action is broadcast over tensor entries.

A unary Symbolica function whose action is broadcast over tensor entries.

```
class BroadcastFunction:
```

Requires features: `python-stubs`

`__new__`

Kind: `AssociatedFunction`

```
def __new__(cls, name: str, *, is_symmetric: Optional[bool] = None, is_antisymmetric:
Optional[bool] = None, is_cyclesymmetric: Optional[bool] = None, is_linear:
Optional[bool] = None, is_flat: Optional[bool] = None, is_scalar: Optional[bool] =
None, is_real: Optional[bool] = None, is_integer: Optional[bool] = None, is_positive:
Optional[bool] = None, tags: Optional[Sequence[str]] = None, aliases:
Optional[Sequence[str]] = None, normalization: Optional[Transformer |
Callable[[Expression], Expression]] = None, print: Optional[Any] = None, derivative:
Optional[Any] = None, series: Optional[Any] = None, eval: Optional[Any] = None, data:
Optional[Expression | int | Float | int | float | str | decimal.Decimal |
ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float |
int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal] |
str | dict | list | bytes] = None) -> BroadcastFunction:
```

Register a Symbolica function for elementwise application to tensor entries.

The function is always tagged as a Spenso broadcast and cannot also be a tensor head.

`name`

Kind: `Parameter`

```
str
```

`is_symmetric`

Kind: `Parameter`

```
Optional[bool]
```

Default: `None`

`is_antisymmetric`

Kind: `Parameter`

```
Optional[bool]
```

Default: `None`

`is_cyclesymmetric`

Kind: `Parameter`

```
Optional[bool]
```

Default: None

is_linear
Kind: Parameter

Optional[bool]

Default: None

is_flat
Kind: Parameter

Optional[bool]

Default: None

is_scalar
Kind: Parameter

Optional[bool]

Default: None

is_real
Kind: Parameter

Optional[bool]

Default: None

is_integer
Kind: Parameter

Optional[bool]

Default: None

is_positive
Kind: Parameter

Optional[bool]

Default: None

tags
Kind: Parameter

Optional[Sequence[str]]

Default: None

aliases
Kind: Parameter

Optional[Sequence[str]]

Default: None

normalization
Kind: Parameter

Optional[Transformer | Callable[[Expression], Expression]]

Default: None

print
Kind: Parameter

Optional[Any]

Default: None

derivative
Kind: Parameter

Optional[Any]

Default: None

series
Kind: Parameter

Optional[Any]

Default: None

eval
Kind: Parameter

Optional[Any]

Default: None

data
Kind: Parameter

Optional[Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal] | str | dict | list | bytes]

Default: None

conj
Kind: AssociatedFunction

def conj() -> BroadcastFunction:

Return Spenso's registered complex-conjugation broadcast function.

__repr__
Kind: Method

def __repr__(self) -> str:

__str__
Kind: Method

def __str__(self) -> str:

to_expression
Kind: Method

def to_expression(self) -> Expression:

Return the registered function as a Symbolica variable expression.

has_tag
Kind: Method

def has_tag(self, tag: str) -> bool:

Whether the registered function carries `tag`.

Unqualified names also match tags registered in Symbolica's Python namespace.

tag
Kind: Parameter

str

get_tags
Kind: Method

def get_tags(self) -> list[str]:

Return the fully qualified tags attached to the registered function.

__call__
Kind: Method

__call__
Kind: Overload

def __call__(self, arg: Tensor | TensorNetwork) -> TensorNetwork:

Apply this unary function lazily to a concrete tensor or tensor network.

arg
Kind: Parameter

Tensor | TensorNetwork

__call__
Kind: Overload

def __call__(self, arg: TensorExpression) -> TensorExpression:

Apply this unary function elementwise, preserving a structured interface.

arg
Kind: Parameter

TensorExpression

__call__
Kind: Overload

def __call__(self, arg: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]) -> Expression:

Apply this unary function to an ordinary scalar expression.

arg
Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

Exhaustive reference · Source

CompiledTensorEvaluator

```
A compiled and optimized evaluator for maximum performance tensor evaluation.
```

```
A compiled and optimized evaluator for maximum performance tensor evaluation.

This class wraps a compiled C++ shared library for extremely fast numerical
evaluation of tensor expressions. It only supports complex-valued evaluation
as this is the most general case.

A compiled and optimized evaluator for maximum performance tensor evaluation.

This class wraps a compiled C++ shared library for extremely fast numerical
evaluation of tensor expressions. It only supports complex-valued evaluation
as this is the most general case.

Create instances using the `TensorEvaluator.compile()` method.

Examples
--------
>>> compiled = evaluator.compile("eval_func", "code.cpp", "lib")
>>> results = compiled.evaluate_complex(large_input_batch)
```

```
class CompiledTensorEvaluator:
```

Requires features: `python-stubs`

evaluate_complex

Kind: `Method`

```
def evaluate_complex(self, inputs: Sequence[Sequence[complex]]) -> list[Tensor]:
```

```
Evaluate the tensor expression for multiple complex-valued parameter inputs.

Uses the compiled C++ code for maximum performance evaluation with complex numbers.

Parameters
----------
inputs : list of list of complex
    List of parameter value lists, where each inner list contains
    complex values for all parameters in the same order as specified
    when creating the original evaluator

Returns
-------
list of Tensor
    List of evaluated tensors, one for each input parameter set

Examples
--------
>>> complex_inputs = [
...     [1.0+2.0j, 3.0+0.0j],
...     [0.0+1.0j, 2.0+1.0j]
```

```
... ]
>>> results = compiled_evaluator.evaluate_complex(complex_inputs)
```

inputs

Kind: Parameter

```
Sequence[Sequence[complex]]
```

```
List of parameter value lists, where each inner list contains complex values for all
parameters in the same order as specified when creating the original evaluator
```

Exhaustive reference · Source

DisplaySettings

```
Presentation settings shared by Typst source, HTML, and SVG rendering.
```


```
Presentation settings shared by Typst source, HTML, and SVG rendering.
```


```
class DisplaySettings:
```

Requires features: `python-stubs`

tensor_layout

Kind: Getter

```
def tensor_layout(self) -> str:
```

show_dimensions

Kind: Getter

```
def show_dimensions(self) -> bool:
```

parentheses

Kind: Getter

```
def parentheses(self) -> bool:
```

commas

Kind: Getter

```
def commas(self) -> Optional[bool]:
```

symbol_scripts

Kind: Getter

```
def symbol_scripts(self) -> bool:
```

index_gap

Kind: Getter

```
def index_gap(self) -> str:
```

factor_gap

Kind: Getter

```
def factor_gap(self) -> str:
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, tensor_layout: str = 'ports', show_dimensions: bool = False,
parentheses: bool = True, commas: Optional[bool] = None, symbol_scripts: bool = True,
index_gap: str = '0.08em', factor_gap: str = '0.12em') -> DisplaySettings:
```

tensor_layout

Kind: Parameter

`str`

Default: `'ports'`

show_dimensions

Kind: Parameter

`bool`

Default: `False`

parentheses

Kind: Parameter

`bool`

Default: `True`

commas

Kind: Parameter

`Optional[bool]`

Default: `None`

symbol_scripts

Kind: Parameter

`bool`

Default: `True`

index_gap

Kind: Parameter

`str`

Default: `'0.08em'`

factor_gap

Kind: Parameter

`str`

Default: `'0.12em'`

ports

Kind: AssociatedFunction

```
def ports() -> DisplaySettings:
```

schoonschip

Kind: AssociatedFunction

```
def schoonschip() -> DisplaySettings:
```

call

Kind: AssociatedFunction

```
def call() -> DisplaySettings:
```

__repr__

Kind: Method

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

PortPattern

```
A Symbolica expression representing one tensor port in a rewrite pattern.
```

```
A Symbolica expression representing one tensor port in a rewrite pattern.

Port patterns may name an exact representation or constrain a wildcard
representation head by its Spenso duality tags. Omitting `index` produces a
stripped representation, suitable for compact vector and trace patterns.

Examples
--------
>>> import symbolica as sp
>>> from symbolica.community.spenso import PortPattern, Representation
>>> D_, i_ = sp.S("D_", "i_")
>>> exact = PortPattern.exact(Representation.mink(4), i_)
>>> generic = PortPattern.self_dual("R_", D_, i_)
>>> stripped = PortPattern.dualizable("C_", D_)
```

```
class PortPattern(Expression):
```

Requires features: python-stubs

exact

Kind: AssociatedFunction

```
def exact(rep: Representation, index: Optional[Expression | int | Float | int | float
| str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal
| complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]] = None) -> PortPattern:
```

```
Match one exact representation, optionally carrying `index`.
```

rep

Kind: Parameter

```
Representation
```

index

Kind: Parameter

```
Optional[Expression | int | Float | int | float | str | decimal.Decimal |
ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float |
int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]]
```

Default: None

any

Kind: AssociatedFunction

```
def any(name: str, dimension: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], index: Optional[Expression | int | Float | int | float | str
| decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]] = None) -> PortPattern:
```

Match any Spenso representation head.

`name` is the reusable Symbolica wildcard name and must end in one
underscore, for example `"R_"`.

name

Kind: Parameter

str

dimension

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]

index

Kind: Parameter

Optional[Expression | int | Float | int | float | str | decimal.Decimal |
ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float |
int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]]

Default: None

self_dual

Kind: AssociatedFunction

```
def self_dual(name: str, dimension: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], index: Optional[Expression | int | Float | int | float | str
| decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]] = None) -> PortPattern:
```

Match a self-dual representation head.

name

Kind: Parameter

str

dimension

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]

index

Kind: Parameter

```
Optional[Expression | int | Float | int | float | str | decimal.Decimal |
ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float |
int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]]
```

Default: None

dualizable

Kind: AssociatedFunction

```
def dualizable(name: str, dimension: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], index: Optional[Expression | int | Float | int | float | str
| decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]] = None, *, dual: bool = False) -> PortPattern:
```

Match a dualizable representation in its base or dual orientation.

name

Kind: Parameter

```
str
```

dimension

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

index

Kind: Parameter

```
Optional[Expression | int | Float | int | float | str | decimal.Decimal |
ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float |
int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]]
```

Default: None

dual

Kind: Parameter

```
bool
```

Default: False

Exhaustive reference · Source

Representation

A representation in the sense of group representation theory for tensor indices.

A representation in the sense of group representation theory for tensor indices.

Representations define the transformation properties of tensor indices under group operations.
They specify the dimension and duality structure, determining which indices can contract.

```
Key concepts:
- **Self-dual**: Indices can contract with other indices of the same representation
- **Dualizable**: Indices can only contract with their dual representation
- **Dimension**: Size of the representation space

Predefined representations are available as class methods:
- `Representation.euc(d)`: Euclidean space (self-dual)
- `Representation.mink(d)`: Minkowski space (self-dual)
- `Representation.bis(d)`: Bispinor (self-dual)
- `Representation.cof(d)`: Color fundamental (dualizable)
- `Representation.coad(d)`: Color adjoint (self-dual)
- `Representation.cos(d)`: Color sextet (dualizable)

# Examples:
```python
from symbolica.community.spenso import Representation

# Standard representations
euclidean = Representation.euc(4)      # 4D Euclidean
lorentz = Representation.mink(4)       # 4D Minkowski
color = Representation.cof(3)          # SU(3) fundamental
adjoint = Representation.coad(8)       # SU(3) adjoint

# Custom representation
custom = Representation("MyRep", 5, is_self_dual=True)

# Create slots with indices
mu_slot = euclidean('mu')              # Euclidean index μ
a_slot = color('a')                    # Color index a

# Generate metric tensors
metric = euclidean.g('mu', 'nu')       # g_μν
```

class Representation:
```

Requires features: `python-stubs`

`dimension`

Kind: `Getter`

```
def dimension(self) -> Expression:
```

Return the dimension carried by this representation.

`__eq__`

Kind: `Method`

```
def __eq__(self, other: object) -> bool:
```

`other`

Kind: `Parameter`

```
object
```

`__call__`

Kind: `Method`

__call__

Kind: Overload

```
def __call__(self, aind: int | Expression | str) -> Slot:
```

```
Create a slot from this representation, by specifying an index.

Parameters
----------
aind : int, str, or Symbol
    The index specification

Returns
-------
Slot
    A new Slot object with the specified index

Examples
--------
>>> from symbolica.community.spenso import Representation
>>> import symbolica as sp
>>> rep = Representation.euc(3)
>>> slot1 = rep('mu')
>>> slot2 = rep(1)
>>> slot3 = rep(sp.S('nu'))
```

aind

Kind: Parameter

```
int | Expression | str
```

```
The index specification
```

__call__

Kind: Overload

```
def __call__(self, aind: Expression) -> Expression | Slot:
```

```
Create a slot or symbolic expression from this representation.

Parameters
----------
aind : Expression
    The index specification (Expression creates symbolic representation)

Returns
-------
Expression
    A symbolic expression representing this representation

Examples
--------
>>> from symbolica.community.spenso import Representation
>>> import symbolica as sp
>>> rep = Representation.euc(3)
>>> expr = rep(sp.E("cos(x)"))
```

aind

Kind: Parameter

Expression

The index specification (Expression creates symbolic representation)

__new__

Kind: AssociatedFunction

def __new__(cls, name: str, dimension: int | Expression | str, is_self_dual: bool = True) -> Representation:

Create and register a new representation with specified properties.

Parameters:
- name: String name for the representation
- dimension: Size of the representation (int or symbolic)
- is_self_dual: If True, creates self-dual representation; if False, creates dualizable pair

Examples:
```python
from symbolica.community.spenso import Representation
import symbolica as sp

# Self-dual representation (indices contract with themselves)
euclidean = Representation("Euclidean", 4, is_self_dual=True)

# Dualizable representation (needs dual partner for contraction)
vector_up = Representation("VectorUp", 4, is_self_dual=False)

# Symbolic dimension
n = sp.S('n')
general = Representation("General", n, is_self_dual=True)
```

name

Kind: Parameter

str

String name for the representation

dimension

Kind: Parameter

int | Expression | str

Size of the representation (int or symbolic)

is_self_dual

Kind: Parameter

bool

Default: True

If True, creates self-dual representation; if False, creates dualizable pair

dual

Kind: Method

def dual(self) -> Representation:

Return the representation paired with this one under index contraction.

casimir

Kind: Method

```
def casimir(self, degree: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal] = 2) -> Expression:
```

Build the degree-k Casimir eigenvalue for this representation.

degree

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

Default: 2

dynkin_index

Kind: Method

```
def dynkin_index(self, degree: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal] = 2) -> Expression:
```

Build the degree-k Dynkin index for this representation.

degree

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

Default: 2

gram

Kind: Method

```
def gram(self, degree: Expression | int | Float | int | float | str | decimal.Decimal
| ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float
| int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal],
other: Optional[Representation] = None) -> Expression:
```

Build a degree-k Gram invariant with another representation.

If `other` is omitted, both sides use this representation.

degree

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

other

Kind: Parameter

Optional[Representation]

Default: None

g

Kind: Method

def g(self, i: int | Expression | str, j: int | Expression | str) ->
TensorExpression:

Create a metric tensor for this representation.

Parameters:
- i: First index
- j: Second index

Examples:
```python
rep = Representation.mink(4)
metric = rep.g('mu', 'nu')  # Minkowski metric g_μν
```

i

Kind: Parameter

int | Expression | str

First index

j

Kind: Parameter

int | Expression | str

Second index

flat

Kind: Method

def flat(self, i: int | Expression | str, j: int | Expression | str) ->
TensorExpression:

Create a musical isomorphism tensor for this representation.

Parameters:
- i: First index
- j: Second index

Examples:
```python
rep = Representation.mink(4)
flat = rep.flat('mu', 'nu')  # Flat isomorphism ♭_μν
```

i

Kind: Parameter

int | Expression | str

First index

j

Kind: Parameter

```
int | Expression | str
```

Second index

id

Kind: Method

```
def id(self, i: int | Expression | str, j: int | Expression | str) ->
TensorExpression:
```

Create an identity tensor for this representation.

```
# Parameters:
- i: First index
- j: Second index

# Examples:
```python
rep = Representation.cof(3)
identity = rep.id('a', 'b')  # Color identity δ_ab
```
```

i

Kind: Parameter

```
int | Expression | str
```

First index

j

Kind: Parameter

```
int | Expression | str
```

Second index

__repr__

Kind: Method

```
def __repr__(self) -> str:
```

__str__

Kind: Method

```
def __str__(self) -> str:
```

to_expression

Kind: Method

```
def to_expression(self) -> Expression:
```

Convert the representation to a symbolic expression.

bis

Kind: AssociatedFunction

```
def bis(dimension: int | Expression | str) -> Representation:
```

Create a bispinor representation.

```
# Parameters:
- dimension: The dimension of the bispinor space
```

dimension

Kind: Parameter

```
int | Expression | str
```

The dimension of the bispinor space

euc

Kind: AssociatedFunction

```
def euc(dimension: int | Expression | str) -> Representation:
```

Create a Euclidean space representation.

```
# Parameters:
- dimension: The dimension of the Euclidean space
```

dimension

Kind: Parameter

```
int | Expression | str
```

The dimension of the Euclidean space

mink

Kind: AssociatedFunction

```
def mink(dimension: int | Expression | str) -> Representation:
```

Create a Minkowski space representation.

```
# Parameters:
- dimension: The dimension of the Minkowski space
```

dimension

Kind: Parameter

```
int | Expression | str
```

The dimension of the Minkowski space

cof

Kind: AssociatedFunction

```
def cof(dimension: int | Expression | str) -> Representation:
```

Create a color fundamental representation.

```
# Parameters:
- dimension: The dimension of the color group (e.g., 3 for SU(3))
```

dimension

Kind: Parameter

```
int | Expression | str
```

The dimension of the color group (e.g., 3 for SU(3))

coad

Kind: AssociatedFunction

```
def coad(dimension: int | Expression | str) -> Representation:
```

Create a color adjoint representation.

```
# Parameters:
- dimension: The dimension of the adjoint representation (e.g., 8 for SU(3))
```

dimension

Kind: Parameter

```
int | Expression | str
```

The dimension of the adjoint representation (e.g., 8 for SU(3))

cos

Kind: AssociatedFunction

```
def cos(dimension: int | Expression | str) -> Representation:
```

Create a color sextet representation.

```
# Parameters:
- dimension: The dimension of the sextet representation (e.g., 6 for SU(3))
```

dimension

Kind: Parameter

```
int | Expression | str
```

The dimension of the sextet representation (e.g., 6 for SU(3))

Exhaustive reference · Source

Slot

A tensor index slot combining a representation with an abstract index.

A tensor index slot combining a representation with an abstract index.

Slots are the building blocks for tensor structures, pairing a representation
(which defines transformation properties) with an abstract index identifier.
Slots with matching representations and indices can be contracted.

Examples:
```python
from symbolica.community.spenso import Slot, Representation
import symbolica as sp

# Create representation and slots
rep = Representation.euc(3)
slot1 = rep('mu')           # Slot with string index
slot2 = rep(1)              # Slot with integer index
slot3 = rep(sp.S('nu'))     # Slot with symbolic index

# Create custom slot
custom_slot = Slot("MyRep", 4, 'alpha', dual=False)

# Use in tensor structures
from symbolica.community.spenso import TensorName
tensor_expression = TensorName("T")(slot1, slot2)
```

class Slot:

Requires features: python-stubs

__eq__
Kind: Method

def __eq__(self, other: object) -> bool:

other
Kind: Parameter

object

__repr__
Kind: Method

def __repr__(self) -> str:

__str__
Kind: Method

def __str__(self) -> str:

dual
Kind: Method

def dual(self) -> Slot:

Return this slot with its representation replaced by the dual representation.

__new__
Kind: AssociatedFunction

def __new__(cls, name: str, dimension: int, aind: int | Expression | str, dual: bool = False) -> Slot:

Create a new slot with a custom representation and index.

Parameters:
- name: String name for the representation
- dimension: Size of the representation space
- aind: The abstract index (int, str, or Symbol)
- dual: If True, creates dualizable representation; if False, self-dual

Examples:
```python
from symbolica.community.spenso import Slot
import symbolica as sp

# Self-dual slot
euclidean_slot = Slot("Euclidean", 4, 'mu', dual=False)

# Dualizable slot
vector_slot = Slot("Vector", 4, 'nu', dual=True)

# With symbolic index
sym_index = sp.S('alpha')
symbolic_slot = Slot("Custom", 3, sym_index, dual=False)
```

name

Kind: Parameter

str

String name for the representation

dimension

Kind: Parameter

int

Size of the representation space

aind

Kind: Parameter

int | Expression | str

The abstract index (int, str, or Symbol)

dual

Kind: Parameter

bool

Default: False

If True, creates dualizable representation; if False, self-dual

to_expression

Kind: Method

def to_expression(self) -> Expression:

Convert the slot to a symbolic expression.

Examples:
```python
rep = Representation.euc(3)
slot = rep('mu')
expr = slot.to_expression()  # Symbolic representation of the slot
```

Exhaustive reference · Source

Tensor

A tensor class that can be either dense or sparse with flexible data types.

A tensor class that can be either dense or sparse with flexible data types.

The tensor can store data as floats, complex numbers, or symbolic expressions.
Tensors have an associated structure that defines their shape and index properties.

Examples

```
>>> from symbolica.community.spenso import Tensor, TensorName, Representation
>>> rep = Representation.euc(4)
>>> structure = TensorName.vector("v")(rep("mu"))
>>> data = [1.0, 2.0, 3.0, 4.0]
>>> tensor = Tensor.dense(structure, data)
>>> sparse_tensor = Tensor.sparse(structure, float)
```

```
class Tensor:
```

Requires features: `python-stubs`

structure

Kind: `Method`

```
def structure(self) -> TensorExpression:
```

```
Return the exact structured expression describing this tensor's data.
```

with_name

Kind: `Method`

```
def with_name(self, name: TensorName | str | Expression | TensorExpression) ->
Tensor:
```

```
Return a copy with a new data identity while preserving its expression and interface.
```

name

Kind: `Parameter`

```
TensorName | str | Expression | TensorExpression
```

sparse

Kind: `AssociatedFunction`

```
def sparse(structure: TensorExpression, type_info: type) -> Tensor:
```

```
Create a new sparse empty tensor with the given structure and data type.

Parameters
----------
structure : TensorExpression
    A named expression with unresolved, explicit, or mixed interface ports
type_info : type
    The data type - either `float` or `Expression` class

Returns
-------
Tensor
    A new sparse tensor with all elements initially zero

Examples
--------
>>> import symbolica
>>> from symbolica.community.spenso import Tensor, TensorName, Representation
>>> rep = Representation.euc(3)
>>> structure = TensorName("T")(rep, rep)
>>> sparse_float = Tensor.sparse(structure, float)
>>> sparse_sym = Tensor.sparse(structure, symbolica.Expression)
```

structure

Kind: `Parameter`

```
TensorExpression
```

```
A named expression with unresolved, explicit, or mixed interface ports
```

type_info

Kind: `Parameter`

type

The data type - either `float` or `Expression` class

dense

Kind: AssociatedFunction

```
def dense(structure: TensorExpression, data: Sequence[Expression] | Sequence[float] |
Sequence[complex]) -> Tensor:
```

Create a new dense tensor with the given structure and data.

```
Parameters
----------
structure : TensorExpression
    A named expression with unresolved, explicit, or mixed interface ports
data : list of float, complex, or Expression
    Tensor values in logical row-major order: the last interface axis varies
    fastest, regardless of the canonical order used by internal storage.
    Representation-axis movement occurs only at this input boundary; later
    index-only storage permutations do not change the public layout.

Returns
-------
Tensor
    A new dense tensor with the specified data

Examples
--------
>>> from symbolica import S
>>> from symbolica.community.spenso import Tensor, TensorName, Representation
>>> rep = Representation.euc(2)
>>> structure = TensorName("T")(rep("mu"), rep("nu"))
>>> data = [1.0, 2.0, 3.0, 4.0]
>>> tensor = Tensor.dense(structure, data)
>>> x, y = S("x", "y")
>>> sym_data = [x, y, x * y, x + y]
>>> sym_tensor = Tensor.dense(structure, sym_data)
```

structure

Kind: Parameter

TensorExpression

A named expression with unresolved, explicit, or mixed interface ports

data

Kind: Parameter

Sequence[Expression] | Sequence[float] | Sequence[complex]

Tensor values in logical row-major order: the last interface axis varies fastest, regardless of the canonical order used by internal storage. Representation-axis movement occurs only at this input boundary; later index-only storage permutations do not change the public layout.

to_dense

Kind: Method

```
def to_dense(self) -> None:
```

Convert this tensor to dense storage format.

Converts sparse tensors to dense format in-place. Dense tensors are unchanged.
This allocates memory for all tensor elements.

Examples

>>> from symbolica.community.spenso import Tensor, TensorName, Representation
>>> rep = Representation.euc(4)
>>> structure = TensorName.vector("v")(rep("mu"))
>>> tensor = Tensor.sparse(structure, float)
>>> tensor[0] = 1.0
>>> tensor.to_dense()

to_sparse

Kind: Method

def to_sparse(self) -> None:

Convert this tensor to sparse storage format.

Converts dense tensors to sparse format in-place, only storing non-zero elements.
This can save memory for tensors with many zero elements.

Examples

>>> from symbolica.community.spenso import Tensor, TensorName, Representation
>>> rep = Representation.euc(2)
>>> structure = TensorName("T")(rep("mu"), rep("nu"))
>>> data = [1.0, 0.0, 0.0, 2.0]
>>> tensor = Tensor.dense(structure, data)
>>> tensor.to_sparse()

__repr__

Kind: Method

def __repr__(self) -> str:

__str__

Kind: Method

def __str__(self) -> str:

format_tensor

Kind: Method

def format_tensor(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None) -> str:

Format this concrete tensor using compact Spenso notation.

Values are read in logical interface order without transposing or densifying
the stored tensor. Large results show a bounded edge preview.

show_dimensions

Kind: Parameter

Optional[bool]

Default: None

settings

Kind: Parameter

Optional[DisplaySettings]

Default: None

to_typst

Kind: Method

```
def to_typst(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None) -> str:
```

Format this concrete tensor as Typst math source.

Values are read in logical interface order without transposing or densifying the stored tensor. Large results show a bounded edge preview.

show_dimensions

Kind: Parameter

Optional[bool]

Default: None

settings

Kind: Parameter

Optional[DisplaySettings]

Default: None

formatted

Kind: Method

```
def formatted(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None, notation_source: Optional[str] = None) ->
FormattedOutput:
```

Build Symbolica's rich display wrapper for this concrete tensor.

Its text, HTML, and LaTeX representations all use logical interface order.

show_dimensions

Kind: Parameter

Optional[bool]

Default: None

settings

Kind: Parameter

Optional[DisplaySettings]

Default: None

notation_source

Kind: Parameter

Optional[str]

Default: None

to_html

Kind: Method

```
def to_html(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None, notation_source: Optional[str] = None) -> str:
```

Compile this concrete tensor to semantic HTML with the optional Typst renderer.

show_dimensions

Kind: Parameter

```
Optional[bool]
```

Default: None

settings

Kind: Parameter

```
Optional[DisplaySettings]
```

Default: None

notation_source

Kind: Parameter

```
Optional[str]
```

Default: None

to_svg

Kind: Method

```
def to_svg(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None, notation_source: Optional[str] = None) -> str:
```

Compile this concrete tensor to SVG with the optional Typst renderer.

show_dimensions

Kind: Parameter

```
Optional[bool]
```

Default: None

settings

Kind: Parameter

```
Optional[DisplaySettings]
```

Default: None

notation_source

Kind: Parameter

```
Optional[str]
```

Default: None

_repr_html_

Kind: Method

```
def _repr_html_(self) -> Optional[str]:
```

_repr_latex_
Kind: Method

```
def _repr_latex_(self) -> Optional[str]:
```

_repr_pretty_
Kind: Method

```
def _repr_pretty_(self, pretty: Any, cycle: bool) -> None:
```

pretty
Kind: Parameter

```
Any
```

cycle
Kind: Parameter

```
bool
```

__len__
Kind: Method

```
def __len__(self) -> int:
```

__getitem__
Kind: Method

__getitem__
Kind: Overload

```
def __getitem__(self, item: slice | int | list[int]) -> Any:
Return tensor data in logical interface order.

An integer is a flat logical row-major position. A list supplies one coordinate
per slot in `structure().interface`; slices likewise traverse flat logical order.
Canonical storage-axis order is never exposed through this API.
```

item
Kind: Parameter

```
slice | int | list[int]
```

__getitem__
Kind: Overload

```
def __getitem__(self, item: slice) -> list[Expression | complex | float]:
Get tensor elements at the specified range of indices.

Slices traverse the flat logical row-major order shown by
`Tensor.structure().interface`;
canonical storage-axis order is not exposed.

Parameters
----------
item : slice
```

```
    Slice object defining the range of indices

Returns
-------
list of float, complex, or Expression
    The tensor elements at the specified range
```

item

Kind: Parameter

```
slice

Slice object defining the range of indices
```

__getitem__

Kind: Overload

```
def __getitem__(self, item: Sequence[int] | int) -> Expression | complex | float:

Get tensor element at the specified index or indices.

Integers are flat logical row-major positions. Coordinate lists follow
`Tensor.structure().interface`; canonical storage-axis order is not exposed.

Parameters
----------
item : int or list of int
    Index specification (int for flat index, list of int for coordinates)

Returns
-------
float, complex, or Expression
    The tensor element at the specified index
```

item

Kind: Parameter

```
Sequence[int] | int

Index specification (int for flat index, list of int for coordinates)
```

__setitem__

Kind: Method

__setitem__

Kind: Overload

```
def __setitem__(self, item: Any, value: Any) -> None:

Set tensor element(s) at the specified index or indices.

Parameters
----------
item : int or list of int
    Logical index specification (int for flat row-major position, list of int
    for coordinates following `structure().interface`)
value : float, complex, or Expression
    The value to set. Its coefficient kind must match the tensor: `float` for
    real storage, `complex` for complex storage, or `Expression` for parametric
storage.
```

```
Examples
--------
>>> from symbolica.community.spenso import Tensor, TensorName, Representation
>>> rep = Representation.euc(2)
>>> structure = TensorName("T")(rep("mu"), rep("nu"))
>>> tensor = Tensor.sparse(structure, float)
>>> tensor[0] = 4.0
>>> tensor[1, 1] = 1.0
```

item

Kind: Parameter

Any

```
Logical index specification (int for flat row-major position, list of int for
coordinates following `structure().interface`)
```

value

Kind: Parameter

Any

```
The value to set. Its coefficient kind must match the tensor: `float` for real
storage, `complex` for complex storage, or `Expression` for parametric storage.
```

__setitem__

Kind: Overload

```
def __setitem__(self, item: int | Sequence[int], value: Expression | complex | float)
-> None:
```

```
Set tensor element at the specified index.

Integers are flat logical row-major positions. Coordinate lists follow
`Tensor.structure().interface`; canonical storage-axis order is not exposed.

Parameters
----------
item : int or list of int
    Index specification (int for flat index, list of int for coordinates)
value : float, complex, or Expression
    The value to set. Use float for real storage, complex for complex storage,
    or Expression for parametric storage.

Examples
--------
>>> from symbolica.community.spenso import Tensor, TensorName, Representation
>>> rep = Representation.euc(2)
>>> structure = TensorName("T")(rep("mu"), rep("nu"))
>>> tensor = Tensor.sparse(structure, float)
>>> tensor[0] = 1.0
>>> tensor[1, 1] = 2.0
```

item

Kind: Parameter

```
int | Sequence[int]
```

```
Index specification (int for flat index, list of int for coordinates)
```

value

Kind: Parameter

```
Expression | complex | float
```

The value to set. Use float for real storage, complex for complex storage, or Expression for parametric storage.

evaluator

Kind: Method

```
def evaluator(self, constants: Mapping[Expression, Expression], funs:
Mapping[tuple[Expression, str, Sequence[Expression]], Expression], params:
Sequence[Expression], iterations: int = 100, n_cores: int = 4, verbose: bool = False)
-> TensorEvaluator:
```

Create an optimized evaluator for symbolic tensor expressions.

Compiles the symbolic expressions in this tensor into an optimized evaluation tree that can efficiently compute numerical values for different parameter inputs.

Parameters
---
constants : dict
    Dict mapping symbolic expressions to their constant numerical values
funs : dict
    Dict mapping function signatures to their symbolic definitions
params : list of Expression
    List of symbolic parameters that will be varied during evaluation
iterations : int, optional
    Number of optimization iterations for Horner scheme (default: 100)
n_cores : int, optional
    Number of CPU cores to use for optimization (default: 4)
verbose : bool, optional
    Whether to print optimization progress (default: False)

Returns

TensorEvaluator
    An optimized evaluator for efficient numerical evaluation

Examples

```
>>> from symbolica import S
>>> from symbolica.community.spenso import Tensor, TensorName, Representation
>>> x, y = S("x", "y")
>>> rep = Representation.euc(2)
>>> structure = TensorName.vector("v")(rep("mu"))
>>> tensor = Tensor.dense(structure, [x * y, x + y])
>>> evaluator = tensor.evaluator(constants={}, funs={}, params=[x, y], iterations=50)
>>> results = evaluator.evaluate_complex([[1.0, 2.0], [3.0, 4.0]])
```

constants

Kind: Parameter

```
Mapping[Expression, Expression]
```

Dict mapping symbolic expressions to their constant numerical values

funs

Kind: Parameter

Mapping[tuple[Expression, str, Sequence[Expression]], Expression]

Dict mapping function signatures to their symbolic definitions

params

Kind: Parameter

Sequence[Expression]

List of symbolic parameters that will be varied during evaluation

iterations

Kind: Parameter

int

Default: 100

Number of optimization iterations for Horner scheme (default: 100)

n_cores

Kind: Parameter

int

Default: 4

Number of CPU cores to use for optimization (default: 4)

verbose

Kind: Parameter

bool

Default: False

Whether to print optimization progress (default: False)

scalar

Kind: Method

```
def scalar(self) -> Expression:
```

Extract the scalar value from a rank-0 (scalar) tensor.

```
Returns
-------
Expression
    The scalar expression contained in this tensor

Raises
------
RuntimeError
    If the tensor is not a scalar

Examples
--------
>>> from symbolica.community.spenso import Tensor, TensorName, Representation
>>> rep = Representation.euc(2)
```

```
>>> p = TensorName.vector("p")
>>> structure = (p(rep("mu")) * p(rep("mu"))).with_name(TensorName("p2"))
>>> scalar_tensor = Tensor.dense(structure, [1.0])
>>> value = scalar_tensor.scalar()
```

index

Kind: Method

```
def index(self, *indices: Any, cook_indices: bool = False) -> TensorNetwork:
```

Reference this tensor with new abstract indices and return a lazy network.

indices

Kind: Parameter

```
Any
```

cook_indices

Kind: Parameter

```
bool
```

Default: `False`

__call__

Kind: Method

```
def __call__(self, *indices: Any, cook_indices: bool = False) -> TensorNetwork:
```

Reference this tensor with new abstract indices and return a lazy network.

indices

Kind: Parameter

```
Any
```

cook_indices

Kind: Parameter

```
bool
```

Default: `False`

__neg__

Kind: Method

```
def __neg__(self) -> TensorNetwork:
```

__add__

Kind: Method

```
def __add__(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

rhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

__radd__

Kind: Method

```
def __radd__(self, lhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

lhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

__sub__

Kind: Method

```
def __sub__(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

rhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

__rsub__

Kind: Method

```
def __rsub__(self, lhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

lhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

__mul__

Kind: Method

```
def __mul__(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

rhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

__rmul__

Kind: Method

```
def __rmul__(self, lhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

lhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

__truediv__

Kind: Method

```
def __truediv__(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression
| TensorNetwork | Tensor) -> TensorNetwork:
```

rhs

Kind: Parameter

Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor

__rtruediv__

Kind: Method

```
def __rtruediv__(self, lhs: Expression | int | Float | ComplexFloat |
TensorExpression | TensorNetwork | Tensor) -> TensorNetwork:
```

lhs

Kind: Parameter

Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor

outer

Kind: Method

```
def outer(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

Form a lazy outer product without contracting compatible ports.

rhs

Kind: Parameter

Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor

contract

Kind: Method

```
def contract(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor, *, left: int, right: int) -> TensorNetwork:
```

Contract one selected pair of ordered interface positions.

rhs

Kind: Parameter

Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor

left

Kind: Parameter

int

right

Kind: Parameter

int

compose

Kind: Method

```
def compose(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor, *, left: tuple[int, int], right: tuple[int, int]) ->
TensorNetwork:
```

Compose two selected `(input, output)` matrix channels.

rhs

Kind: Parameter

Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor

left

Kind: Parameter

```
tuple[int, int]
```

right

Kind: Parameter

```
tuple[int, int]
```

dot

Kind: Method

```
def dot(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

```
Contract two rank-one operands into the canonical dot form.
```

rhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

trace

Kind: Method

```
def trace(self, *, channel: Optional[tuple[int, int]] = None) -> TensorNetwork:
```

```
Close `channel`, or the unique matrix channel when it is omitted.
```

channel

Kind: Parameter

```
Optional[tuple[int, int]]
```

Default: `None`

__iter__

Kind: Method

```
def __iter__(self) -> Iterator[Any]:
```

```
Iterator
```

Exhaustive reference · Source

TensorEvaluator

```
An optimized evaluator for symbolic tensor expressions.
```

```
An optimized evaluator for symbolic tensor expressions.

This class provides efficient numerical evaluation of symbolic tensor expressions
after optimization. It supports both real and complex-valued evaluations.

Create instances using the `Tensor.evaluator()` method rather than directly.

Examples
--------
>>> evaluator = my_tensor.evaluator(constants={}, funs={}, params=[x, y])
>>> results = evaluator.evaluate([[1.0, 2.0], [3.0, 4.0]])
```

```
class TensorEvaluator:
```

Requires features: `python-stubs`

evaluate

Kind: `Method`

```
def evaluate(self, inputs: Sequence[Sequence[float]]) -> list[Tensor]:
```

```
Evaluate the tensor expression for multiple real-valued parameter inputs.

Parameters
----------
inputs : list of list of float
    List of parameter value lists, where each inner list contains
    numerical values for all parameters in the same order as specified
    when creating the evaluator

Returns
-------
list of Tensor
    List of evaluated tensors, one for each input parameter set

Raises
------
ValueError
    If the evaluator contains complex coefficients

Examples
--------
>>> results = evaluator.evaluate([[1.0, 2.0], [3.0, 4.0]])
```

inputs

Kind: `Parameter`

```
Sequence[Sequence[float]]
```

```
List of parameter value lists, where each inner list contains numerical values for
all parameters in the same order as specified when creating the evaluator
```

evaluate_complex

Kind: `Method`

```
def evaluate_complex(self, inputs: Sequence[Sequence[complex]]) -> list[Tensor]:
```

```
Evaluate the expression for multiple inputs and return the results.
```

inputs

Kind: `Parameter`

```
Sequence[Sequence[complex]]
```

compile

Kind: `Method`

```
def compile(self, function_name: str, filename: str, library_name: str, inline_asm:
str = 'default', optimization_level: int = 3, compiler_path: Optional[str] = None,
custom_header: Optional[str] = None) -> CompiledTensorEvaluator:
```

```
Compile the evaluator to a shared library using C++ for maximum performance.

Generates optimized C++ code with optional inline assembly and compiles it
```

```
into a shared library that can be loaded for extremely fast evaluation.

Parameters
----------
function_name : str
    Name for the generated C++ function
filename : str
    Path for the generated C++ source file
library_name : str
    Name for the compiled shared library
inline_asm : str, optional
    Type of inline assembly optimization ("default", "x64", "aarch64", "none")
optimization_level : int, optional
    Compiler optimization level 0-3 (default: 3)
compiler_path : str, optional
    Path to specific C++ compiler (default: system default)
custom_header : str, optional
    Additional C++ header code to include

Returns
-------
CompiledTensorEvaluator
    A compiled evaluator for maximum performance evaluation

Examples
--------
>>> compiled = evaluator.compile(
...     function_name="fast_eval",
...     filename="tensor_eval.cpp",
...     library_name="tensor_lib",
...     optimization_level=3,
... )
>>> results = compiled.evaluate_complex([[1.0, 2.0], [3.0, 4.0]])
```

function_name

Kind: Parameter

str

Name for the generated C++ function

filename

Kind: Parameter

str

Path for the generated C++ source file

library_name

Kind: Parameter

str

Name for the compiled shared library

inline_asm

Kind: Parameter

str

Default: `'default'`

Type of inline assembly optimization ("default", "x64", "aarch64", "none")

`optimization_level`
Kind: `Parameter`

`int`

Default: `3`

Compiler optimization level 0-3 (default: 3)

`compiler_path`
Kind: `Parameter`

`Optional[str]`

Default: `None`

Path to specific C++ compiler (default: system default)

`custom_header`
Kind: `Parameter`

`Optional[str]`

Default: `None`

Additional C++ header code to include

Exhaustive reference · Source

`TensorExpression`
A Symbolica expression with an ordered external tensor interface.

A Symbolica expression with an ordered external tensor interface.

Predefined tensors are constructed by typed factories and indexed afterward
in logical interface order. Dimensions are representation metadata, not
scalar tensor arguments.

Examples

```
>>> from symbolica.community.spenso import Representation, TensorExpression
>>> mink = Representation.mink(4)
>>> metric = TensorExpression.g(mink)("mu", "nu")
>>> flat = TensorExpression.flat(mink)("mu", "nu")
>>> gamma = TensorExpression.gamma(4)("i", "j", "mu")
>>> gamma5 = TensorExpression.gamma5(4)("i", "j")
>>> projm = TensorExpression.projm(4)("i", "j")
>>> projp = TensorExpression.projp(4)("i", "j")
>>> sigma = TensorExpression.sigma(4)("mu", "nu", "i", "j")
>>> structure_constant = TensorExpression.f(8)("a", "b", "c")
>>> generator = TensorExpression.t(8, 3)("a", "i", "j")
```

`class TensorExpression(Expression):`

Requires features: `python-stubs`

`rank`
Kind: `Getter`

def rank(self) -> int:

Number of external tensor ports in the ordered interface.

is_scalar

Kind: Getter

def is_scalar(self) -> bool:

Whether the expression has no external tensor ports.

interface

Kind: Getter

def interface(self) -> tuple:

Ordered external interface as concrete `Slot` objects or unresolved
`Representation`s.

name

Kind: Getter

def name(self) -> Optional[TensorName]:

The optional identity used when this expression describes stored data.

g

Kind: AssociatedFunction

def g(rep: Representation) -> TensorExpression:

Create an unresolved metric tensor for `rep`.

Call the result with two indices to fill its ports in logical order.

rep

Kind: Parameter

Representation

flat

Kind: AssociatedFunction

def flat(rep: Representation) -> TensorExpression:

Create an unresolved musical-isomorphism tensor for `rep`.

Call the result with two indices to fill its ports in logical order.

rep

Kind: Parameter

Representation

gamma

Kind: AssociatedFunction

def gamma(minkowski_dimension: int | Expression | str) -> TensorExpression:

Create an unresolved gamma matrix with Minkowski dimension
`minkowski_dimension`.

Its public ports use storage order: bispinor, bispinor, then Minkowski.

The bispinor dimension is four. Call the result with `(i, j, mu)` to index it.

minkowski_dimension

Kind: Parameter

int | Expression | str

gamma5

Kind: AssociatedFunction

def gamma5(spinor_dimension: int | Expression | str) -> TensorExpression:

Create an unresolved gamma-five matrix with two bispinor ports of `spinor_dimension`; call the result with `(i, j)`.

spinor_dimension

Kind: Parameter

int | Expression | str

projm

Kind: AssociatedFunction

def projm(spinor_dimension: int | Expression | str) -> TensorExpression:

Create an unresolved left-chiral projector with two bispinor ports of `spinor_dimension`; call the result with `(i, j)`.

spinor_dimension

Kind: Parameter

int | Expression | str

projp

Kind: AssociatedFunction

def projp(spinor_dimension: int | Expression | str) -> TensorExpression:

Create an unresolved right-chiral projector with two bispinor ports of `spinor_dimension`; call the result with `(i, j)`.

spinor_dimension

Kind: Parameter

int | Expression | str

sigma

Kind: AssociatedFunction

def sigma(minkowski_dimension: int | Expression | str) -> TensorExpression:

Create an unresolved sigma matrix with Minkowski dimension `minkowski_dimension`.

Its logical ports are two Minkowski ports followed by two four-dimensional bispinor ports. Call the result with `(mu, nu, i, j)` to index it.

minkowski_dimension

Kind: Parameter

int | Expression | str

f

Kind: AssociatedFunction

```
def f(adjoint_dimension: int | Expression | str) -> TensorExpression:
```

Create an unresolved color structure constant.

All three logical ports are adjoint representations of
`adjoint_dimension`; call the result with `(a, b, c)`.

adjoint_dimension

Kind: Parameter

```
int | Expression | str
```

t

Kind: AssociatedFunction

```
def t(adjoint_dimension: int | Expression | str, fundamental_dimension: int |
Expression | str) -> TensorExpression:
```

Create an unresolved color generator.

The logical ports are adjoint, fundamental, and antifundamental. Dimensions
belong to those representations and are not tensor key arguments.

```
Examples
--------
>>> from symbolica.community.spenso import TensorExpression
>>> generator = TensorExpression.t(8, 3)
>>> indexed = generator("a", "i", "j")
```

adjoint_dimension

Kind: Parameter

```
int | Expression | str
```

fundamental_dimension

Kind: Parameter

```
int | Expression | str
```

with_name

Kind: Method

```
def with_name(self, name: TensorName | str | Expression | TensorExpression) ->
TensorExpression:
```

Return a named descriptor without changing the underlying expression.

name

Kind: Parameter

```
TensorName | str | Expression | TensorExpression
```

__len__

Kind: Method

```
def __len__(self) -> int:
```

to_expression

Kind: Method

```
def to_expression(self) -> Expression:
```

Drop the structured interface and return an ordinary Symbolica `Expression`.

reinfer

Kind: Method

```
def reinfer(self) -> TensorExpression:
```

Re-parse the underlying symbolic expression and rebuild its ordered tensor interface.

expand

Kind: Method

```
def expand(self, var: Optional[Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]] = None, via_poly: Optional[bool] = None) -> TensorExpression:
```

Expand scalar algebra while preserving and validating the tensor interface.

var

Kind: Parameter

```
Optional[Expression | int | Float | int | float | str | decimal.Decimal |
ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float |
int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]]
```

Default: None

via_poly

Kind: Parameter

```
Optional[bool]
```

Default: None

simplify_gamma

Kind: Method

```
def simplify_gamma(self, settings: Optional[GammaSimplifySettings] = None) ->
TensorExpression:
```

Apply Idenso's gamma-algebra simplifier and re-infer the tensor interface.

settings

Kind: Parameter

```
Optional[GammaSimplifySettings]
```

Default: None

collect_gamma_chains

Kind: Method

```
def collect_gamma_chains(self) -> TensorExpression:
```

Convert bispinor tensors into chain/trace shorthands and join adjacent gamma chains.

simplify_gamma0

Kind: Method

```
def simplify_gamma0(self) -> TensorExpression:
```

Simplify products and linear combinations involving the time-like gamma matrix `gamma0`.

simplify_gamma_conjugate

Kind: Method

```
def simplify_gamma_conjugate(self) -> TensorExpression:
```

Rewrite conjugated gamma matrices as gamma0-sandwiched matrices.

simplify_epsilon

Kind: Method

```
def simplify_epsilon(self) -> TensorExpression:
```

Simplify Levi-Civita/metric contractions and pairs of Levi-Civita tensors.

simplify_metrics

Kind: Method

```
def simplify_metrics(self) -> TensorExpression:
```

Contract metric and identity tensors and re-infer the external interface.

simplify_color

Kind: Method

```
def simplify_color(self, settings: Optional[ColorSimplifySettings] = None) ->
TensorExpression:
```

Apply Idenso's SU(N) color-algebra simplifier and re-infer the interface.

settings

Kind: Parameter

```
Optional[ColorSimplifySettings]
```

Default: None

collect_color

Kind: Method

```
def collect_color(self) -> TensorExpression:
```

Factor around tensors carrying fundamental, antifundamental, or adjoint color.

collect_color_constants

Kind: Method

```
def collect_color_constants(self) -> TensorExpression:
```

Factor around recognized scalar color invariants such as Casimirs and indices.

to_color_casimir

Kind: Method

```
def to_color_casimir(self, *, fundamental: Representation, adjoint: Representation,
settings: Optional[ColorCasimirSettings] = None) -> TensorExpression:
```

Rewrite supplied color dimensions and invariants into a representation-aware Casimir basis.

fundamental
Kind: Parameter

Representation

adjoint
Kind: Parameter

Representation

settings
Kind: Parameter

Optional[ColorCasimirSettings]

Default: None

to_cof_dimension_invariants
Kind: Method

def to_cof_dimension_invariants(self) -> TensorExpression:

Replace supported `cof(N)` invariants by explicit dimension formulas.

wrap_color
Kind: Method

def wrap_color(self, symbol: Expression) -> TensorExpression:

Expand around color structures and wrap each scalar coefficient with `symbol`.

symbol
Kind: Parameter

Expression

expand_mink
Kind: Method

def expand_mink(self) -> list[tuple[Expression, Expression]]:

Selectively expand around Minkowski structures into `(structure, coefficient)` pairs.

expand_bis
Kind: Method

def expand_bis(self) -> list[tuple[Expression, Expression]]:

Selectively expand around bispinor structures into `(structure, coefficient)` pairs.

expand_mink_bis
Kind: Method

def expand_mink_bis(self) -> list[tuple[Expression, Expression]]:

Selectively expand around Minkowski and bispinor structures into factorized pairs.

expand_metrics
Kind: Method

```
def expand_metrics(self) -> list[tuple[Expression, Expression]]:
```

Selectively expand around metric tensors into `(structure, coefficient)` pairs.

expand_color

Kind: Method

```
def expand_color(self) -> list[tuple[Expression, Expression]]:
```

Selectively expand around color structures into `(structure, coefficient)` pairs.

expand_in_patterns

Kind: Method

```
def expand_in_patterns(self, patterns: Sequence[Expression]) ->
list[tuple[Expression, Expression]]:
```

Selectively expand around the supplied Symbolica patterns into factorized pairs.

patterns

Kind: Parameter

```
Sequence[Expression]
```

wrap_indices

Kind: Method

```
def wrap_indices(self, header: Expression) -> Expression:
```

Wrap every abstract-index payload with `header` and return an ordinary expression.

Wrapped payloads are not Spenso abstract indices until they are cooked, so the result intentionally has no `TensorExpression` interface.

header

Kind: Parameter

```
Expression
```

cook_indices

Kind: Method

```
def cook_indices(self, settings: Optional[CookSettings] = None) -> TensorExpression:
```

Flatten nested representation-index payloads using index cooking by default.

settings

Kind: Parameter

```
Optional[CookSettings]
```

Default: None

cook_function

Kind: Method

```
def cook_function(self, settings: Optional[CookSettings] = None) -> Expression:
```

Encode one function call as an ordinary Symbolica expression.

settings

Kind: Parameter

Optional[CookSettings]

Default: None

wrap_dummies

Kind: Method

def wrap_dummies(self, header: Expression) -> TensorExpression:

Wrap only contracted-index payloads with `header` and re-infer the interface.

Raises `ValueError` when the expression cannot be parsed as a tensor network.

header

Kind: Parameter

Expression

list_dangling

Kind: Method

def list_dangling(self) -> list[Expression]:

Return the ordered external, uncontracted indices as Symbolica expressions.

Raises `ValueError` when the expression cannot be parsed as a tensor network.

canonize

Kind: Method

def canonize(self) -> TensorExpression:

Canonically order tensor factors and deterministically rename contracted indices.

alias_subtensors

Kind: Method

def alias_subtensors(self, tensor_name: str) -> tuple[Expression, list[tuple[Expression, Expression]]]:

Replace nested tensor subexpressions by aliases and return the root plus alias mappings.

tensor_name

Kind: Parameter

str

spenso_conjugate

Kind: Method

def spenso_conjugate(self) -> TensorExpression:

Complex-conjugate this tensor expression and re-infer its interface.

conjugate_transpose

Kind: Method

def conjugate_transpose(self, representation: Representation) -> TensorExpression:

Complex-conjugate this expression and transpose slots in `representation`.

representation

Kind: Parameter

Representation

dirac_adjoint

Kind: Method

```
def dirac_adjoint(self) -> TensorExpression:
```

Construct the physics-aware Dirac adjoint and re-infer the tensor interface.

cook

Kind: Method

```
def cook(self, settings: Optional[CookSettings] = None) -> Expression:
```

Encode this expression using Idenso's reversible cooking format.

settings

Kind: Parameter

Optional[CookSettings]

Default: None

uncook

Kind: Method

```
def uncook(self, settings: Optional[CookSettings] = None) -> TensorExpression:
```

Restore a reversibly cooked expression and re-infer its tensor interface.

settings

Kind: Parameter

Optional[CookSettings]

Default: None

schoonschip

Kind: Method

```
def schoonschip(self, settings: Optional[SchoonschipSettings] = None) ->
TensorExpression:
```

Simplify tensor shorthands using the configured Schoonschip traversal.

settings

Kind: Parameter

Optional[SchoonschipSettings]

Default: None

schoonschip_net

Kind: Method

```
def schoonschip_net(self, settings: Optional[SchoonschipSettings] = None, *,
expand_contracted_sums: bool = False) -> TensorExpression:
```

Parse and contract this expression as a symbolic tensor network.

settings

Kind: Parameter

Optional[SchoonschipSettings]

Default: None

expand_contracted_sums

Kind: Parameter

bool

Default: False

to_dots

Kind: Method

def to_dots(self) -> TensorExpression:

Convert contracted rank-one tensors into compact dot-product notation.

normalize_dots

Kind: Method

def normalize_dots(self) -> TensorExpression:

Canonicalize compact dot-product shorthands without expanding them.

expand_dots

Kind: Method

def expand_dots(self) -> TensorExpression:

Expand dot products into explicit metric and indexed-vector contractions.

metric_shorthand_to_dot

Kind: Method

def metric_shorthand_to_dot(self) -> TensorExpression:

Replace metric shorthand such as `g(p(rep), q(rep))` by a compact dot product.

undo_all

Kind: Method

def undo_all(self) -> TensorExpression:

Expand every Idenso tensor shorthand into explicit tensor syntax.

undo_schoonschip

Kind: Method

def undo_schoonschip(self) -> TensorExpression:

Expand Schoonschip shorthands while leaving dots, chains, and traces compact.

undo_dots

Kind: Method

def undo_dots(self) -> TensorExpression:

Expand dot-product shorthands while leaving other shorthands compact.

undo_chain

Kind: Method

```
def undo_chain(self) -> TensorExpression:
```

Expand open-chain shorthands while leaving other shorthands compact.

undo_trace

Kind: Method

```
def undo_trace(self) -> TensorExpression:
```

Expand trace shorthands while leaving other shorthands compact.

collect_chains

Kind: Method

```
def collect_chains(self, representation: Representation) -> TensorExpression:
```

Join adjacent open chains for `representation` and re-infer the interface.

representation

Kind: Parameter

Representation

chainify

Kind: Method

```
def chainify(self, representation: Representation) -> TensorExpression:
```

Rewrite tensors with two `representation` slots as open-chain factors.

representation

Kind: Parameter

Representation

normalize_chains

Kind: Method

```
def normalize_chains(self) -> TensorExpression:
```

Convert chains whose endpoints coincide into trace shorthands.

undo_single_length

Kind: Method

```
def undo_single_length(self) -> TensorExpression:
```

Replace one-factor chain shorthands by their underlying tensor factor.

to_network

Kind: Method

```
def to_network(self, library: Optional[TensorLibrary] = None) -> TensorNetwork:
```

Materialize unresolved ports and parse this expression as a tensor network.

When `library` is omitted, use the built-in four-dimensional HEP and SU(3) library.

library

Kind: Parameter

Optional[TensorLibrary]

Default: None

index

Kind: Method

```
def index(self, *indices: Any, cook_indices: bool = False) -> TensorExpression:
```

Fill the unresolved external ports with `indices` in interface order.

Pass `AUTO` to leave a port unresolved. Set `cook_indices=True` to flatten nested symbolic index payloads before insertion. Repeated compatible indices contract their ports, in which case the result no longer carries the original stored-data identity.

indices

Kind: Parameter

Any

cook_indices

Kind: Parameter

bool

Default: False

__call__

Kind: Method

```
def __call__(self, *indices: Any, cook_indices: bool = False) -> TensorExpression:
```

Fill the unresolved external ports with `indices` in interface order.

indices

Kind: Parameter

Any

cook_indices

Kind: Parameter

bool

Default: False

__neg__

Kind: Method

```
def __neg__(self) -> TensorExpression:
```

__add__

Kind: Method

```
def __add__(self, rhs: Any) -> TensorExpression | TensorNetwork:
```

rhs

Kind: Parameter

Any

__radd__

Kind: Method

```
def __radd__(self, lhs: Any) -> TensorExpression | TensorNetwork:
```

lhs

Kind: Parameter

Any

__sub__

Kind: Method

```
def __sub__(self, rhs: Any) -> TensorExpression | TensorNetwork:
```

rhs

Kind: Parameter

Any

__rsub__

Kind: Method

```
def __rsub__(self, lhs: Any) -> TensorExpression | TensorNetwork:
```

lhs

Kind: Parameter

Any

__mul__

Kind: Method

```
def __mul__(self, rhs: Any) -> TensorExpression | TensorNetwork:
```

rhs

Kind: Parameter

Any

__rmul__

Kind: Method

```
def __rmul__(self, lhs: Any) -> TensorExpression | TensorNetwork:
```

lhs

Kind: Parameter

Any

__truediv__

Kind: Method

```
def __truediv__(self, rhs: Any) -> TensorExpression | TensorNetwork:
```

rhs

Kind: Parameter

Any

__rtruediv__

Kind: Method

```
def __rtruediv__(self, lhs: Any) -> TensorExpression | TensorNetwork:
```

lhs

Kind: Parameter

```
Any
```

outer

Kind: Method

```
def outer(self, rhs: Any) -> TensorExpression | TensorNetwork:
```

```
Form an outer product without contracting compatible ports.
```

rhs

Kind: Parameter

```
Any
```

contract

Kind: Method

```
def contract(self, rhs: Any, *, left: int, right: int) -> TensorExpression |
TensorNetwork:
```

```
Contract one selected pair of ordered interface positions.
```

rhs

Kind: Parameter

```
Any
```

left

Kind: Parameter

```
int
```

right

Kind: Parameter

```
int
```

compose

Kind: Method

```
def compose(self, rhs: Any, *, left: tuple[int, int], right: tuple[int, int]) ->
TensorExpression | TensorNetwork:
```

```
Compose two selected `(input, output)` matrix channels.
```

rhs

Kind: Parameter

```
Any
```

left

Kind: Parameter

```
tuple[int, int]
```

right

Kind: Parameter

`tuple[int, int]`

trace

Kind: Method

```
def trace(self, *, channel: Optional[tuple[int, int]] = None) -> TensorExpression:
```

Close `channel`, or the unique matrix channel when it is omitted.

channel

Kind: Parameter

`Optional[tuple[int, int]]`

Default: None

format_tensor

Kind: Method

```
def format_tensor(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None) -> str:
```

Format this structured expression using compact Spenso notation.

show_dimensions

Kind: Parameter

`Optional[bool]`

Default: None

settings

Kind: Parameter

`Optional[DisplaySettings]`

Default: None

to_typst

Kind: Method

```
def to_typst(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None) -> str:
```

Format this structured expression as Typst math source.

show_dimensions

Kind: Parameter

`Optional[bool]`

Default: None

settings

Kind: Parameter

`Optional[DisplaySettings]`

Default: None

formatted

Kind: Method

```
def formatted(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None, notation_source: Optional[str] = None) ->
FormattedOutput:
```

Build Symbolica's rich display value, including semantic HTML when the optional ``gammaloop[typst-display]`` renderer is installed.

``notation_source`` is a trusted complete replacement for the bundled ``notation.typ`` module, not a style fragment. Typst executes it. When HTML cannot be rendered, the result retains its LaTeX and text forms.

show_dimensions

Kind: Parameter

```
Optional[bool]
```

Default: None

settings

Kind: Parameter

```
Optional[DisplaySettings]
```

Default: None

notation_source

Kind: Parameter

```
Optional[str]
```

Default: None

to_html

Kind: Method

```
def to_html(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None, notation_source: Optional[str] = None) -> str:
```

Compile this expression to semantic HTML with the optional Typst renderer.

Install ``gammaloop[typst-display]`` to enable this method. ``notation_source``, when supplied, is trusted Typst code replacing the complete bundled ``notation.typ`` module.

show_dimensions

Kind: Parameter

```
Optional[bool]
```

Default: None

settings

Kind: Parameter

```
Optional[DisplaySettings]
```

Default: None

notation_source

Kind: Parameter

Optional[str]

Default: None

to_svg

Kind: Method

```
def to_svg(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None, notation_source: Optional[str] = None) -> str:
```

Compile this expression to SVG with the optional Typst renderer.

Install ``gammaloop[typst-display]`` to enable this method.
``notation_source``, when supplied, is trusted Typst code replacing the
complete bundled ``notation.typ`` module.

show_dimensions

Kind: Parameter

Optional[bool]

Default: None

settings

Kind: Parameter

Optional[DisplaySettings]

Default: None

notation_source

Kind: Parameter

Optional[str]

Default: None

__repr__

Kind: Method

```
def __repr__(self) -> str:
```

__str__

Kind: Method

```
def __str__(self) -> str:
```

_repr_pretty_

Kind: Method

```
def _repr_pretty_(self, pretty: Any, cycle: bool) -> None:
```

pretty

Kind: Parameter

Any

cycle
Kind: Parameter

bool

_repr_html_
Kind: Method

def _repr_html_(self) -> Optional[str]:

_repr_latex_
Kind: Method

def _repr_latex_(self) -> str:

__getitem__
Kind: Method

__getitem__
Kind: Overload

def __getitem__(self, item: int) -> list[int]:

Convert a logical row-major flat index to tensor coordinates.

item
Kind: Parameter

int

__getitem__
Kind: Overload

def __getitem__(self, item: Sequence[int]) -> int:

Convert tensor coordinates to a logical row-major flat index.

item
Kind: Parameter

Sequence[int]

__getitem__
Kind: Overload

def __getitem__(self, item: slice) -> list[list[int]]:

Expand a slice of logical row-major flat indices to tensor coordinates.

item
Kind: Parameter

slice

Exhaustive reference · Source

TensorFunctionLibrary

A registry of elementwise tensor functions used during network evaluation.

A registry of elementwise tensor functions used during network evaluation.

class TensorFunctionLibrary:

Requires features: `python-stubs`

`__new__`

Kind: `AssociatedFunction`

```
def __new__(cls) -> TensorFunctionLibrary:
```

```
Create a new empty tensor function library.

Initializes an empty library ready for registering tensor functions.

Returns
-------
TensorFunctionLibrary
    A new empty function library

Examples
--------
>>> from symbolica.community.spenso import TensorFunctionLibrary
>>> lib = TensorFunctionLibrary()
```

`register`

Kind: `Method`

```
def register(self, function: BroadcastFunction, callback: Any) -> None:
```

```
Register an elementwise callback for concrete tensor execution.

The callback receives a Python `float` or `complex` matching each input value and
must return either type. The result uses complex storage if any returned value is
complex; otherwise it uses real storage.
```

`function`

Kind: `Parameter`

```
BroadcastFunction
```

`callback`

Kind: `Parameter`

```
Any
```

Exhaustive reference · Source

`TensorLibrary`

```
A library for registering and managing tensor templates and structures.
```

```
A library for registering and managing tensor templates and structures.

The TensorLibrary provides a centralized registry for tensor definitions that can be
reused across tensor networks and expressions. It manages tensor structures with
their
associated names and can resolve symbolic references to registered tensors.

```python
from symbolica.community.spenso import Tensor, TensorLibrary, TensorName,
Representation

lib = TensorLibrary()
rep = Representation.euc(3)
```
```

```
name = TensorName("my_tensor")
structure = name(rep, rep)
tensor = Tensor.dense(structure, [1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 1.0, 0.0, 0.0])
lib.register(tensor)
tensor_ref = lib[name]
```

```
class TensorLibrary:
```

Requires features: `python-stubs`

`__new__`

Kind: `AssociatedFunction`

```
def __new__(cls) -> TensorLibrary:
```

```
Create a new empty tensor library.

Initializes an empty library ready for registering tensor structures.
The library automatically manages internal tensor IDs.

Returns
-------
TensorLibrary
    A new empty tensor library

Examples
--------
>>> from symbolica.community.spenso import TensorLibrary
>>> lib = TensorLibrary()
```

`construct`

Kind: `AssociatedFunction`

```
def construct() -> TensorLibrary:
```

```
Create a new empty tensor library (static method).

Initializes an empty library ready for registering tensor structures.
The library automatically manages internal tensor IDs.

Returns
-------
TensorLibrary
    A new empty tensor library

Examples
--------
>>> from symbolica.community.spenso import TensorLibrary
>>> lib = TensorLibrary.construct()
```

`register`

Kind: `Method`

```
def register(self, tensor: Tensor) -> None:
```

```
Register a tensor in the library.

Adds a tensor template to the library that can be referenced by name
in tensor networks and symbolic expressions. The tensor must have a name
```

```
set in its structure.

Parameters
----------
tensor : Tensor
    A named tensor with a fully unresolved interface, or a named scalar

Examples
--------
>>> from symbolica.community.spenso import Tensor, TensorLibrary, TensorName,
Representation
>>> lib = TensorLibrary()
>>> rep = Representation.euc(3)
>>> name = TensorName("my_tensor")
>>> structure = name(rep, rep)
>>> tensor = Tensor.dense(structure, [1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 1.0, 0.0, 0.0])
>>> lib.register(tensor)
>>> tensor_ref = lib[name]
```

tensor

Kind: Parameter

```
Tensor
```

```
A named tensor with a fully unresolved interface, or a named scalar
```

__getitem__

Kind: Method

```
def __getitem__(self, key: TensorExpression | TensorName | Expression | str) ->
TensorExpression:
```

```
Retrieve a registered tensor structure by name.

Looks up a previously registered tensor by its name and returns
a reference structure that can be used to create new tensor instances.

Parameters
----------
key : TensorExpression, TensorName, Expression, or str
    An exact unresolved tensor signature in registered storage order, or a
    symbol-only convenience key

Returns
-------
TensorExpression
    An atomic reference with the requested exact interface, or the registered
    logical interface for a symbol-only lookup

Raises
------
RuntimeError
    If the tensor name is not found in the library

Examples
--------
>>> exact = lib[TensorName("T")(1, Representation.euc(3))]
>>> unique_by_name = lib["T"]
```

key

Kind: Parameter

```
TensorExpression | TensorName | Expression | str
```

```
An exact unresolved tensor signature in registered storage order, or a symbol-only
convenience key
```

hep_lib

Kind: AssociatedFunction

```
def hep_lib() -> TensorLibrary:
```

```
Create a library pre-loaded with High Energy Physics tensor definitions.

Returns a library containing standard HEP tensors such as gamma matrices,
color generators, metric tensors, and other commonly used structures in
particle physics calculations. They are floating point tensors with f64 precision.

Returns
-------
TensorLibrary
    A TensorLibrary pre-populated with HEP tensor definitions

Examples
--------
>>> import symbolica
>>> from symbolica.community.spenso import TensorLibrary, TensorName
>>> hep_lib = TensorLibrary.hep_lib()
>>> gamma_structure = hep_lib[symbolica.S("spenso::gamma")]
```

hep_lib_atom

Kind: AssociatedFunction

```
def hep_lib_atom() -> TensorLibrary:
```

```
Create a library pre-loaded with High Energy Physics tensor definitions.

Returns a library containing standard HEP tensors such as gamma matrices,
color generators, metric tensors, and other commonly used structures in
particle physics calculations. They are tensors with atom numeric entries.

Returns
-------
TensorLibrary
    A TensorLibrary pre-populated with HEP tensor definitions

Examples
--------
>>> import symbolica
>>> from symbolica.community.spenso import TensorLibrary, TensorName
>>> hep_lib = TensorLibrary.hep_lib_atom()
>>> gamma_structure = hep_lib[symbolica.S("spenso::gamma")]
```

Exhaustive reference · Source

TensorName

```
A symbolic name for tensor expressions.
```

```
A symbolic name for tensor expressions.

TensorName represents named tensor functions that can be called with scalar
arguments, slots,
and representations to create tensor expressions. Names can have various mathematical
properties like symmetry,
antisymmetry, and custom normalization or printing behavior.

The predefined accessors such as `TensorName.gamma()` and `TensorName.t()`
return raw fixed heads for introspection and pattern construction; those
reserved names cannot be called directly. Use the matching
`TensorExpression` factory for concrete tensors or `TensorPattern` shortcut
for rewrite patterns.

Examples
--------
>>> from symbolica.community.spenso import TensorName, Slot, Representation
>>> T = TensorName("T")
>>> symmetric_T = TensorName("S", is_symmetric=True)
>>> antisymmetric_T = TensorName("A", is_antisymmetric=True)
>>> rep = Representation.cof(3)
>>> mu = rep('mu')
>>> nu = rep('nu')
>>> tensor_expression = T(mu, nu)
```

```
class TensorName:
```

Requires features: `python-stubs`

`__new__`

Kind: `AssociatedFunction`

```
def __new__(cls, name: str, *, rank: Optional[int] = None, is_symmetric:
Optional[bool] = None, is_antisymmetric: Optional[bool] = None, is_cyclesymmetric:
Optional[bool] = None, is_linear: Optional[bool] = None, is_flat: Optional[bool] =
None, is_scalar: Optional[bool] = None, is_real: Optional[bool] = None, is_integer:
Optional[bool] = None, is_positive: Optional[bool] = None, tags:
Optional[Sequence[str]] = None, aliases: Optional[Sequence[str]] = None,
normalization: Optional[Transformer | Callable[[Expression], Expression]] = None,
print: Optional[Any] = None, derivative: Optional[Any] = None, series: Optional[Any]
= None, eval: Optional[Any] = None, data: Optional[Expression | int | Float | int |
float | str | decimal.Decimal | ComplexFloat | Float | int | float | str |
decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float
| int | float | str | decimal.Decimal] | str | dict | list | bytes] = None) ->
TensorName:
```

```
Create a new tensor name with optional mathematical properties.

Parameters
----------
name : str
    The string name for the tensor function
is_symmetric : bool, optional
    If True, tensor is symmetric under index permutation
is_antisymmetric : bool, optional
    If True, tensor is antisymmetric under index permutation
is_cyclesymmetric : bool, optional
    If True, tensor is symmetric under cyclic permutations
```

```
is_linear : bool, optional
    If True, tensor is linear in its arguments
rank : int, optional
    The declared rank. Only rank one has a dedicated construction invariant.
tags : list[str], optional
    Extra Symbolica tags. The Spenso tensor tag is always included.
normalization, print, derivative, series, eval, data : optional
    Symbolica symbol callbacks and metadata.

Returns
-------
TensorName
    A new TensorName with the specified properties

Examples
--------
>>> from symbolica.community.spenso import TensorName
>>> T = TensorName("T")
>>> g = TensorName("g", is_symmetric=True)
>>> F = TensorName("F", is_antisymmetric=True)
>>> D = TensorName("D", is_linear=True)
```

name

Kind: Parameter

str

The string name for the tensor function

rank

Kind: Parameter

Optional[int]

Default: None

The declared rank. Only rank one has a dedicated construction invariant.

is_symmetric

Kind: Parameter

Optional[bool]

Default: None

If True, tensor is symmetric under index permutation

is_antisymmetric

Kind: Parameter

Optional[bool]

Default: None

If True, tensor is antisymmetric under index permutation

is_cyclesymmetric

Kind: Parameter

Optional[bool]

Default: None

If True, tensor is symmetric under cyclic permutations

is_linear
Kind: Parameter

Optional[bool]

Default: None

If True, tensor is linear in its arguments

is_flat
Kind: Parameter

Optional[bool]

Default: None

is_scalar
Kind: Parameter

Optional[bool]

Default: None

is_real
Kind: Parameter

Optional[bool]

Default: None

is_integer
Kind: Parameter

Optional[bool]

Default: None

is_positive
Kind: Parameter

Optional[bool]

Default: None

tags
Kind: Parameter

Optional[Sequence[str]]

Default: None

Extra Symbolica tags. The Spenso tensor tag is always included.

aliases
Kind: Parameter

Optional[Sequence[str]]

Default: None

normalization

Kind: Parameter

Optional[Transformer | Callable[[Expression], Expression]]

Default: None

Symbolica symbol callbacks and metadata.

print

Kind: Parameter

Optional[Any]

Default: None

Symbolica symbol callbacks and metadata.

derivative

Kind: Parameter

Optional[Any]

Default: None

Symbolica symbol callbacks and metadata.

series

Kind: Parameter

Optional[Any]

Default: None

Symbolica symbol callbacks and metadata.

eval

Kind: Parameter

Optional[Any]

Default: None

Symbolica symbol callbacks and metadata.

data

Kind: Parameter

Optional[Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal] | str | dict | list | bytes]

Default: None

Symbolica symbol callbacks and metadata.

vector

Kind: AssociatedFunction

def vector(name: str, *, is_symmetric: Optional[bool] = None, is_antisymmetric: Optional[bool] = None, is_cyclesymmetric: Optional[bool] = None, is_linear: Optional[bool] = None, is_flat: Optional[bool] = None, is_scalar: Optional[bool] =

None, is_real: Optional[bool] = None, is_integer: Optional[bool] = None, is_positive: Optional[bool] = None, tags: Optional[Sequence[str]] = None, aliases: Optional[Sequence[str]] = None, normalization: Optional[Transformer | Callable[[Expression], Expression]] = None, print: Optional[Any] = None, derivative: Optional[Any] = None, series: Optional[Any] = None, eval: Optional[Any] = None, data: Optional[Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal] | str | dict | list | bytes] = None) -> TensorName:

Create a rank-one tensor name.

name

Kind: Parameter

str

is_symmetric

Kind: Parameter

Optional[bool]

Default: None

is_antisymmetric

Kind: Parameter

Optional[bool]

Default: None

is_cyclesymmetric

Kind: Parameter

Optional[bool]

Default: None

is_linear

Kind: Parameter

Optional[bool]

Default: None

is_flat

Kind: Parameter

Optional[bool]

Default: None

is_scalar

Kind: Parameter

Optional[bool]

Default: None

is_real

Kind: Parameter

Optional[bool]

Default: None

is_integer
Kind: Parameter

Optional[bool]

Default: None

is_positive
Kind: Parameter

Optional[bool]

Default: None

tags
Kind: Parameter

Optional[Sequence[str]]

Default: None

aliases
Kind: Parameter

Optional[Sequence[str]]

Default: None

normalization
Kind: Parameter

Optional[Transformer | Callable[[Expression], Expression]]

Default: None

print
Kind: Parameter

Optional[Any]

Default: None

derivative
Kind: Parameter

Optional[Any]

Default: None

series
Kind: Parameter

Optional[Any]

Default: None

eval
Kind: Parameter

Optional[Any]

Default: None

data

Kind: Parameter

Optional[Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal] | str | dict | list | bytes]

Default: None

__repr__

Kind: Method

```
def __repr__(self) -> str:
```

__str__

Kind: Method

```
def __str__(self) -> str:
```

to_expression

Kind: Method

```
def to_expression(self) -> Expression:
```

Convert the tensor name to a symbolic expression.

```
Returns
-------
Expression
    A symbolic Expression representing this tensor name

Examples
--------
>>> T = TensorName("T")
>>> expr = T.to_expression()
```

has_tag

Kind: Method

```
def has_tag(self, tag: str) -> bool:
```

Check whether this tensor name carries `tag`.

tag

Kind: Parameter

str

get_tags

Kind: Method

```
def get_tags(self) -> list[str]:
```

Return all Symbolica tags carried by this tensor name.

g

Kind: AssociatedFunction

```
def g() -> TensorName:
```

Predefined metric tensor name.

flat

Kind: AssociatedFunction

```
def flat() -> TensorName:
```

Predefined musical isomorphism tensor name. This enables dualizing self dual indices.

gamma

Kind: AssociatedFunction

```
def gamma() -> TensorName:
```

Predefined gamma matrix name for introspection and pattern construction.

The matching typed factories use storage order: bispinor-in,
bispinor-out, then Minkowski.

gamma5

Kind: AssociatedFunction

```
def gamma5() -> TensorName:
```

Predefined gamma5 matrix name.

projm

Kind: AssociatedFunction

```
def projm() -> TensorName:
```

Predefined left chiral projector name.

projp

Kind: AssociatedFunction

```
def projp() -> TensorName:
```

Predefined right chiral projector name.

sigma

Kind: AssociatedFunction

```
def sigma() -> TensorName:
```

Predefined sigma matrix name.

f

Kind: AssociatedFunction

```
def f() -> TensorName:
```

Predefined color structure constant name.

t

Kind: AssociatedFunction

```
def t() -> TensorName:
```

Predefined color generator name.

__call__

Kind: Method

```
def __call__(self, *args: Slot | Representation | Expression | int | Float | int |
float | str | decimal.Decimal | ComplexFloat | Float | int | float | str |
decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float
| int | float | str | decimal.Decimal]) -> TensorExpression:
```

```
Call the tensor name with arguments to create a tensor expression.

Accepts scalar key expressions followed by any mix of slots and representations.

Parameters
----------
*args : Expression, Slot, or Representation
    Scalar key expressions followed by structural ports

Returns
-------
TensorExpression
    A structured expression with explicit and/or unresolved ports

Examples
--------
>>> from symbolica.community.spenso import TensorName, Slot, Representation
>>> import symbolica as sp
>>> T = TensorName("T")
>>> rep = Representation.euc(3)
>>> mu = rep("mu")
>>> tensor = T(mu, rep)
```

args

Kind: Parameter

```
Slot | Representation | Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]
```

Exhaustive reference · Source

TensorNetwork

```
A graph of tensor operations that can be simplified and executed.
```

```
A graph of tensor operations that can be simplified and executed.

Named tensor expressions are resolved through a `TensorLibrary`. Register concrete
data
before constructing and executing a network; an expression alone supplies structure,
not
component values.

A network retains the semantic source expression and its public tensor interface
separately from the executable graph and its stored values. Value specialization
and graph execution therefore do not rewrite the source expression returned by
`structure()` or used by the semantic display methods.

Examples
```

```
--------
>>> from symbolica.community.spenso import (
...     ExecutionMode,
...     Representation,
...     Tensor,
...     TensorLibrary,
...     TensorName,
...     TensorNetwork,
... )
>>> rep = Representation.euc(2)
>>> A = TensorName("A")
>>> structure = A(rep, rep)
>>> library = TensorLibrary()
>>> library.register(
...     Tensor.dense(structure, [1.0, 0.0, 0.0, 1.0])
... )
>>> network = TensorNetwork(
...     A(rep("i"), rep("j")),
...     library=library,
... )
>>> network.execute(library=library, mode=ExecutionMode.All)
>>> result = network.result_tensor(library=library)
>>> len(result)
4
```

```
class TensorNetwork:
```

Requires features: `python-stubs`

`__new__`

Kind: `AssociatedFunction`

```
def __new__(cls, expr: Any, library: Optional[TensorLibrary] = None) ->
TensorNetwork:
```

```
Create a tensor network by parsing an arithmetic expression.

Parses symbolic expressions containing tensor operations and converts them
into an optimizable computational graph representation.

Parameters
----------
expr : ArithmeticStructure
    The arithmetic expression or tensor structure to parse
library : TensorLibrary, optional
    Tensor library for resolving named tensor references. Defaults to the built-in
    four-dimensional HEP and SU(3) library returned by `TensorLibrary.hep_lib()`.

Returns
-------
TensorNetwork
    A new TensorNetwork representing the parsed expression
```

`expr`

Kind: `Parameter`

```
Any
```

```
The arithmetic expression or tensor structure to parse
```

library

Kind: Parameter

```
Optional[TensorLibrary]
```

Default: None

Tensor library for resolving named tensor references. Defaults to the built-in four-dimensional HEP and SU(3) library returned by `TensorLibrary.hep_lib()`.

one

Kind: AssociatedFunction

```
def one() -> TensorNetwork:
```

Create a tensor network representing the scalar value 1.

```
Returns
-------
TensorNetwork
    A TensorNetwork containing only the scalar 1

Examples
--------
>>> from symbolica.community.spenso import TensorNetwork
>>> one_net = TensorNetwork.one()
>>> result = one_net.result_scalar()
```

bracket

Kind: AssociatedFunction

```
def bracket() -> Expression:
```

Return the symbolic head used for structured product brackets.

broadcast

Kind: AssociatedFunction

```
def broadcast(str: str) -> Expression:
```

Create a Symbolica function symbol tagged for elementwise tensor broadcasting.

str

Kind: Parameter

```
str
```

zero

Kind: AssociatedFunction

```
def zero() -> TensorNetwork:
```

Create a tensor network representing the scalar value 0.

```
Returns
-------
TensorNetwork
    A TensorNetwork containing only the scalar 0

Examples
--------
```

```
>>> from symbolica.community.spenso import TensorNetwork
>>> zero_net = TensorNetwork.zero()
>>> result = zero_net.result_scalar()
```

replace

Kind: Method

```
def replace(self, pattern: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], rhs: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal] | HeldExpression | Callable[[dict[Expression, Expression]],
Expression] | int | float | complex | Float | ComplexFloat | decimal.Decimal, cond:
Optional[PatternRestriction | Condition] = None, non_greedy_wildcards:
Optional[Sequence[Expression]] = None, level_range: Optional[tuple[int,
Optional[int]]] = None, level_is_tree_depth: Optional[bool] = None,
allow_new_wildcards_on_rhs: Optional[bool] = None, rhs_cache_size: Optional[int] =
None, repeat: Optional[bool] = None) -> TensorNetwork:
```

```
Replace patterns in stored symbolic network values.

Rewrites scalar coefficients and symbolic elements of parametric tensors in
the execution store. Tensor identities, graph topology, the public interface,
and the semantic source expression returned by `structure()` are unchanged.
Rewrite a `TensorExpression` before constructing the network when the source
tensor expression itself should change.

Parameters
----------
pattern : Expression
    The symbolic pattern to match within stored values
rhs : Expression
    The replacement expression or pattern
cond : PatternRestriction or Condition, optional
    Additional restriction that each match must satisfy
non_greedy_wildcards : list of Expression, optional
    List of wildcard symbols to match non-greedily
level_range : tuple of int, optional
    Tuple specifying depth range for pattern matching
level_is_tree_depth : bool, optional
    Whether level refers to tree depth or expression depth
allow_new_wildcards_on_rhs : bool, optional
    Allow new wildcards in replacement pattern
rhs_cache_size : int, optional
    Size of cache for replacement pattern compilation
repeat : bool, optional
    Whether to repeatedly apply the replacement until no more matches

Returns
-------
TensorNetwork
    A new TensorNetwork with matching stored values replaced and the same
    semantic source structure
```

pattern

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

The symbolic pattern to match within stored values

rhs

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal] | HeldExpression | Callable[[dict[Expression, Expression]], Expression] | int | float | complex | Float | ComplexFloat | decimal.Decimal

The replacement expression or pattern

cond

Kind: Parameter

Optional[PatternRestriction | Condition]

Default: None

Additional restriction that each match must satisfy

non_greedy_wildcards

Kind: Parameter

Optional[Sequence[Expression]]

Default: None

List of wildcard symbols to match non-greedily

level_range

Kind: Parameter

Optional[tuple[int, Optional[int]]]

Default: None

Tuple specifying depth range for pattern matching

level_is_tree_depth

Kind: Parameter

Optional[bool]

Default: None

Whether level refers to tree depth or expression depth

allow_new_wildcards_on_rhs

Kind: Parameter

Optional[bool]

Default: None

Allow new wildcards in replacement pattern

rhs_cache_size

Kind: Parameter

Optional[int]

Default: None

Size of cache for replacement pattern compilation

repeat

Kind: Parameter

Optional[bool]

Default: None

Whether to repeatedly apply the replacement until no more matches

evaluate

Kind: Method

```
def evaluate(self, constants: Mapping[Expression, float], functions:
Mapping[Expression, Any]) -> TensorNetwork:
```

Evaluate symbolic tensor values in the execution store.

Substitutes symbolic constants and functions in stored parametric tensor elements, converting them to concrete numerical tensors. Tensor identities, graph topology, the public interface, and the semantic source expression are retained.

```
Parameters
----------
constants : dict
    Dict mapping symbolic expressions to their numerical values
functions : dict
    Dict mapping function symbols to Python callable objects

Returns
-------
TensorNetwork
    A new TensorNetwork with stored tensor values evaluated and the same
    semantic source structure
```

constants

Kind: Parameter

Mapping[Expression, float]

Dict mapping symbolic expressions to their numerical values

functions

Kind: Parameter

Mapping[Expression, Any]

Dict mapping function symbols to Python callable objects

execute

Kind: Method

```
def execute(self, library: Optional[TensorLibrary] = None, function_library:
Optional[TensorFunctionLibrary] = None, n_steps: Optional[int] = None, mode:
ExecutionMode = ExecutionMode.All) -> None:
```

Execute the tensor network to perform tensor contractions and simplifications.

Processes the computational graph by executing tensor operations such as contractions, additions, and multiplications. The execution can be controlled by mode and step limits.

```
Parameters
----------
library : TensorLibrary, optional
    Optional tensor library for resolving tensor operations
function_library : TensorFunctionLibrary or None, optional
    Tensor function callbacks; None uses the built-in function library
n_steps : int, optional
    Maximum number of execution steps (None for complete execution)
mode : ExecutionMode, optional
    Execution strategy. ExecutionMode.Single selects one smallest-degree rewrite per
    step; use n_steps to bound how many steps run.

Examples
--------
>>> from symbolica.community.spenso import TensorNetwork, ExecutionMode,
TensorLibrary
>>> network = TensorNetwork(some_expression)
>>> network.execute()
>>> network.execute(n_steps=5)
>>> network.execute(mode=ExecutionMode.Scalar)
>>> lib = TensorLibrary.hep_lib()
>>> network.execute(library=lib)
```

`library`

Kind: Parameter

Optional[TensorLibrary]

Default: None

Optional tensor library for resolving tensor operations

`function_library`

Kind: Parameter

Optional[TensorFunctionLibrary]

Default: None

Tensor function callbacks; None uses the built-in function library

`n_steps`

Kind: Parameter

Optional[int]

Default: None

Maximum number of execution steps (None for complete execution)

mode

Kind: Parameter

ExecutionMode

Default: ExecutionMode.All

Execution strategy. ExecutionMode.Single selects one smallest-degree rewrite per step; use n_steps to bound how many steps run.

result_tensor

Kind: Method

```
def result_tensor(self, library: Optional[TensorLibrary] = None) -> Tensor:
```

Extract the final tensor result from the executed network.

After network execution, retrieves the computed tensor result. The network should be executed before calling this method.

```
Parameters
----------
library : TensorLibrary, optional
    Optional tensor library for resolving tensor structures

Returns
-------
Tensor
    The computed tensor result

Raises
------
RuntimeError
    If the network execution resulted in an error

Examples
--------
>>> from symbolica.community.spenso import TensorNetwork, TensorLibrary
>>> network = TensorNetwork(tensor_expression)
>>> network.execute()
>>> result = network.result_tensor()
>>> lib = TensorLibrary.hep_lib()
>>> result_with_lib = network.result_tensor(library=lib)
```

library

Kind: Parameter

Optional[TensorLibrary]

Default: None

Optional tensor library for resolving tensor structures

result_scalar

Kind: Method

```
def result_scalar(self) -> Expression:
```

Extract the final scalar result from the executed network.

```
For networks that evaluate to scalar expressions, retrieves the computed
scalar value. The network should be executed before calling this method.

Returns
-------
Expression
    The computed scalar expression

Raises
------
RuntimeError
    If the network execution resulted in an error

Examples
--------
>>> from symbolica.community.spenso import TensorNetwork
>>> network = TensorNetwork(scalar_expression)
>>> network.execute()
>>> scalar_result = network.result_scalar()
```

__str__

Kind: Method

```
def __str__(self) -> str:
```

```
Return a DOT representation of the executable network graph.

Generates a DOT format representation of the computational graph that can be
visualized using graphviz or similar tools.
```

to_dot

Kind: Method

```
def to_dot(self) -> str:
```

```
Return the computational graph in Graphviz DOT format.
```

format_tensor

Kind: Method

```
def format_tensor(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None) -> str:
```

```
Format the exact semantic structure using compact Spenso notation.
```

show_dimensions

Kind: Parameter

```
Optional[bool]
```

Default: None

settings

Kind: Parameter

```
Optional[DisplaySettings]
```

Default: None

to_typst

Kind: Method

```
def to_typst(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None) -> str:
```

Format the semantic source structure as Typst math source.

show_dimensions

Kind: Parameter

```
Optional[bool]
```

Default: None

settings

Kind: Parameter

```
Optional[DisplaySettings]
```

Default: None

formatted

Kind: Method

```
def formatted(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None, notation_source: Optional[str] = None) ->
FormattedOutput:
```

Build Symbolica's rich display wrapper for the semantic source structure.

show_dimensions

Kind: Parameter

```
Optional[bool]
```

Default: None

settings

Kind: Parameter

```
Optional[DisplaySettings]
```

Default: None

notation_source

Kind: Parameter

```
Optional[str]
```

Default: None

to_html

Kind: Method

```
def to_html(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None, notation_source: Optional[str] = None) -> str:
```

show_dimensions

Kind: Parameter

```
Optional[bool]
```

Default: None

settings

Kind: Parameter

Optional[DisplaySettings]

Default: None

notation_source

Kind: Parameter

Optional[str]

Default: None

to_svg

Kind: Method

```
def to_svg(self, show_dimensions: Optional[bool] = None, *, settings:
Optional[DisplaySettings] = None, notation_source: Optional[str] = None) -> str:
```

show_dimensions

Kind: Parameter

Optional[bool]

Default: None

settings

Kind: Parameter

Optional[DisplaySettings]

Default: None

notation_source

Kind: Parameter

Optional[str]

Default: None

_repr_html_

Kind: Method

```
def _repr_html_(self) -> Optional[str]:
```

structure

Kind: Method

```
def structure(self) -> TensorExpression:
```

Return the semantic source expression and its public tensor interface.

This expression records the tensor-aware structure used for composition and provenance. It is not reconstructed from the current execution store, so `replace()`, `evaluate()`, and `execute()` leave it unchanged. Use `result_scalar()` or `result_tensor()` to inspect the current computed value.

index

Kind: Method

```
def index(self, *indices: Any, cook_indices: bool = False) -> TensorNetwork:
```

Fill the unresolved external ports with `indices` in interface order.

Pass `AUTO` to leave a port unresolved. Set `cook_indices=True` to flatten nested symbolic index payloads before insertion.

indices

Kind: Parameter

Any

cook_indices

Kind: Parameter

bool

Default: False

__call__

Kind: Method

```
def __call__(self, *indices: Any, cook_indices: bool = False) -> TensorNetwork:
```

Fill the unresolved external ports with `indices` in interface order.

indices

Kind: Parameter

Any

cook_indices

Kind: Parameter

bool

Default: False

__neg__

Kind: Method

```
def __neg__(self) -> TensorNetwork:
```

__add__

Kind: Method

```
def __add__(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

Add two tensor networks element-wise.

```
Parameters
----------
rhs : TensorNetwork
    The tensor network to add (right-hand side)

Returns
-------
TensorNetwork
    A new TensorNetwork representing the sum

Examples
--------
```

```
>>> net1 = TensorNetwork(expr1)
>>> net2 = TensorNetwork(expr2)
>>> sum_net = net1 + net2
```

rhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

The tensor network to add (right-hand side)

__radd__

Kind: Method

```
def __radd__(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

Add two tensor networks element-wise (right-hand addition).

rhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

__sub__

Kind: Method

```
def __sub__(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

Subtract one tensor network from another element-wise.

```
Parameters
----------
rhs : TensorNetwork
    The tensor network to subtract (right-hand side)

Returns
-------
TensorNetwork
    A new TensorNetwork representing the difference

Examples
--------
>>> net1 = TensorNetwork(expr1)
>>> net2 = TensorNetwork(expr2)
>>> diff_net = net1 - net2
```

rhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

The tensor network to subtract (right-hand side)

__rsub__

Kind: Method

```
def __rsub__(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

Subtract one tensor network from another (right-hand subtraction).

rhs

Kind: Parameter

Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor

__mul__

Kind: Method

def __mul__(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor) -> TensorNetwork:

Multiply two tensor networks.

Parameters
----
rhs : TensorNetwork
    The tensor network to multiply with (right-hand side)

Returns

TensorNetwork
    A new TensorNetwork representing the product

Examples

>>> net1 = TensorNetwork(expr1)
>>> net2 = TensorNetwork(expr2)
>>> product_net = net1 * net2

rhs

Kind: Parameter

Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor

The tensor network to multiply with (right-hand side)

__rmul__

Kind: Method

def __rmul__(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor) -> TensorNetwork:

Multiply two tensor networks (right-hand multiplication).

rhs

Kind: Parameter

Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor

__truediv__

Kind: Method

def __truediv__(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor) -> TensorNetwork:

rhs

Kind: Parameter

Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor

__rtruediv__

Kind: Method

```
def __rtruediv__(self, lhs: Expression | int | Float | ComplexFloat |
TensorExpression | TensorNetwork | Tensor) -> TensorNetwork:
```

lhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

outer

Kind: Method

```
def outer(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor) -> TensorNetwork:
```

Form an outer tensor product without contracting compatible ports.

rhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

contract

Kind: Method

```
def contract(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor, *, left: int, right: int) -> TensorNetwork:
```

Contract one selected pair of public interface positions.

rhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

left

Kind: Parameter

```
int
```

right

Kind: Parameter

```
int
```

compose

Kind: Method

```
def compose(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression |
TensorNetwork | Tensor, *, left: tuple[int, int], right: tuple[int, int]) ->
TensorNetwork:
```

Compose two explicitly selected matrix channels.

rhs

Kind: Parameter

```
Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor
```

left

Kind: Parameter

tuple[int, int]

right

Kind: Parameter

tuple[int, int]

dot

Kind: Method

def dot(self, rhs: Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor) -> TensorNetwork:

Contract two rank-one operands into the canonical dot form.

rhs

Kind: Parameter

Expression | int | Float | ComplexFloat | TensorExpression | TensorNetwork | Tensor

trace

Kind: Method

def trace(self, *, channel: Optional[tuple[int, int]] = None) -> TensorNetwork:

Close a selected or uniquely inferred propagation channel.

channel

Kind: Parameter

Optional[tuple[int, int]]

Default: None

Exhaustive reference · Source

TensorPattern

A tagged Symbolica expression intended for tensor rewrite rules.

A tagged Symbolica expression intended for tensor rewrite rules.

`args` contains scalar tensor arguments and `ports` contains structural
syntax. The two named sections are concatenated in that order. Since this is
a pattern expression rather than a concrete tensor, either section may
contain ordinary or sequence-wildcard Symbolica expressions.

Examples

```
>>> import symbolica as sp
>>> from symbolica.community.spenso import (
...     PortPattern, TensorExpression, TensorName, TensorPattern,
... )
>>> k_, D_, mu_, i_, j_, rest___ = sp.S(
...     "k_", "D_", "mu_", "i_", "j_", "rest___",
... )
>>> fixed = TensorPattern(
...     TensorName("A"),
...     args=[k_],
```

```
...     ports=[PortPattern.any("R_", D_, i_), rest___],
... )
>>> generic = TensorPattern.any("T_", ports=[rest___])
>>> target = TensorExpression.gamma(4)("i", "j", "mu").to_expression()
>>> rule = TensorPattern.gamma(D_, i_, j_, mu_)
>>> target.replace(rule, 0)
0
```

```
class TensorPattern(Expression):
```

Requires features: `python-stubs`

`__new__`

Kind: `AssociatedFunction`

```
def __new__(cls, head: TensorName, *, args: Sequence[Expression | int | Float | int |
float | str | decimal.Decimal | ComplexFloat | Float | int | float | str |
decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float
| int | float | str | decimal.Decimal]] = [], ports: Sequence[Slot | Representation |
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]] = []) ->
TensorPattern:
```

```
Create a pattern for the fixed tensor `head`.

Scalar `args` are always emitted before structural `ports`.
```

`head`

Kind: `Parameter`

```
TensorName
```

`args`

Kind: `Parameter`

```
Sequence[Expression | int | Float | int | float | str | decimal.Decimal |
ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float |
int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]]
```

Default: `[]`

`ports`

Kind: `Parameter`

```
Sequence[Slot | Representation | Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]]
```

Default: `[]`

`any`

Kind: `AssociatedFunction`

```
def any(name: str, *, args: Sequence[Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]] = [], ports: Sequence[Slot | Representation | Expression |
int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int |
```

float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]] = []) -> TensorPattern:

Match any tensor-tagged head.

`name` must end in one underscore, for example `"T_"`.

name

Kind: Parameter

str

args

Kind: Parameter

Sequence[Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]]

Default: []

ports

Kind: Parameter

Sequence[Slot | Representation | Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]]

Default: []

vector

Kind: AssociatedFunction

def vector(name: str, *, args: Sequence[Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]] = [], ports: Sequence[Slot | Representation | Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]] = []) -> TensorPattern:

Match any rank-one tensor head.

Pattern construction intentionally does not impose a concrete arity, so
a sequence wildcard may describe the scalar arguments or structural port.

name

Kind: Parameter

str

args

Kind: Parameter

Sequence[Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]]

Default: `[]`

ports

Kind: `Parameter`

```
Sequence[Slot | Representation | Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]]
```

Default: `[]`

g

Kind: `AssociatedFunction`

```
def g(rep_pattern: Representation | Expression, i: Expression | int | Float | int |
float | str | decimal.Decimal | ComplexFloat | Float | int | float | str |
decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float
| int | float | str | decimal.Decimal], j: Expression | int | Float | int | float |
str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]) -> TensorPattern:
```

Match a metric tensor in logical index order.

rep_pattern

Kind: `Parameter`

```
Representation | Expression
```

i

Kind: `Parameter`

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

j

Kind: `Parameter`

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

flat

Kind: `AssociatedFunction`

```
def flat(rep_pattern: Representation | Expression, i: Expression | int | Float | int
| float | str | decimal.Decimal | ComplexFloat | Float | int | float | str |
decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float
| int | float | str | decimal.Decimal], j: Expression | int | Float | int | float |
str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]) -> TensorPattern:
```

Match a musical-isomorphism tensor in logical index order.

rep_pattern

Kind: `Parameter`

Representation | Expression

i

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

j

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

gamma

Kind: AssociatedFunction

def gamma(minkowski_dimension: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal], i: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal], j: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal], mu: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]) -> TensorPattern:

Match a gamma matrix in storage `(i, j, mu)` order.

minkowski_dimension

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

i

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

j

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

mu

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

gamma5

Kind: AssociatedFunction

def gamma5(spinor_dimension: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal], i: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal], j: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]) -> TensorPattern:

Match a gamma-five matrix in logical `(i, j)` order.

spinor_dimension

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

i

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

j

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

projm

Kind: AssociatedFunction

def projm(spinor_dimension: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal], i: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal], j: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]) -> TensorPattern:

Match a left-chiral projector in logical `(i, j)` order.

spinor_dimension

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

i

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

j

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

projp

Kind: AssociatedFunction

def projp(spinor_dimension: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal], i: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal], j: Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]) -> TensorPattern:

Match a right-chiral projector in logical `(i, j)` order.

spinor_dimension

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

i

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

j

Kind: Parameter

Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float | str | decimal.Decimal]

sigma

Kind: AssociatedFunction

```
def sigma(minkowski_dimension: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], mu: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], nu: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], i: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], j: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]) -> TensorPattern:
```

Match a sigma matrix. Arguments use logical `(mu, nu, i, j)` order.

minkowski_dimension

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

mu

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

nu

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

i

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

j

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

f

Kind: AssociatedFunction

```
def f(adjoint_dimension: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
```

```
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], a: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], b: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], c: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]) -> TensorPattern:
```

Match a color structure constant in logical `(a, b, c)` order.

adjoint_dimension

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

a

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

b

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

c

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

t

Kind: AssociatedFunction

```
def t(adjoint_dimension: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], fundamental_dimension: Expression | int | Float | int | float
| str | decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal
| complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], a: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], i: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal], j: Expression | int | Float | int | float | str |
decimal.Decimal | ComplexFloat | Float | int | float | str | decimal.Decimal |
```

```
complex | tuple[Float | int | float | str | decimal.Decimal, Float | int | float |
str | decimal.Decimal]) -> TensorPattern:
```

```
Match a color generator in logical `(a, i, j)` order.
```

adjoint_dimension

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

fundamental_dimension

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

a

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

i

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

j

Kind: Parameter

```
Expression | int | Float | int | float | str | decimal.Decimal | ComplexFloat | Float
| int | float | str | decimal.Decimal | complex | tuple[Float | int | float | str |
decimal.Decimal, Float | int | float | str | decimal.Decimal]
```

Exhaustive reference · Source

ExecutionMode

```
Execution modes for tensor network evaluation.
```

```
Execution modes for tensor network evaluation.

Controls how the tensor network execution engine processes the computational graph.

Variants
--------
Single : Select one smallest-degree rewrite per step; without `n_steps`, continue
until no work remains
Scalar : Only contract scalar operations, leaving tensor structure intact
All : Execute all possible contractions for complete evaluation
```

```
class ExecutionMode(enum.Enum):
```

Requires features: `python-stubs`

`Single`

Kind: `Variant`

`Select one smallest-degree rewrite per step; without `n_steps`, continue until no work remains`

`Scalar`

Kind: `Variant`

`Only contract scalar operations, leaving tensor structure intact`

`All`

Kind: `Variant`

`Execute all possible contractions for complete evaluation`

Exhaustive reference · Source

`SymbolicParallelism`

`Policy for Rayon operations that manipulate Symbolica expressions.`

`Policy for Rayon operations that manipulate Symbolica expressions.`

```
class SymbolicParallelism(enum.Enum):
```

Requires features: `python-stubs`

`Auto`

Kind: `Variant`

`Permit Rayon when licensed and use workload heuristics where available.`

`Serial`

Kind: `Variant`

`Keep symbolic operations on the calling thread.`

`Parallel`

Kind: `Variant`

`Force Rayon without `Auto`'s Symbolica license safety check.`

Exhaustive reference · Source

`as_tensor`

`Restore tensor-aware dispatch after a base Symbolica transformation.`

`Restore tensor-aware dispatch after a base Symbolica transformation.`

```
def as_tensor(expression: Any) -> TensorExpression:
```

Requires features: `python-stubs`

Exhaustive reference · Source

`chain`

`Build an explicitly-ended ordered tensor chain.`

`Build an explicitly-ended ordered tensor chain.`

```
def chain(start_slot: Slot, end_slot: Slot, *factors: Any) -> TensorExpression |
TensorNetwork:
```

Requires features: `python-stubs`

Exhaustive reference · Source

dot

Contract two rank-one tensors into the canonical dot form.

Contract two rank-one tensors into the canonical dot form.

```
def dot(left: Any, right: Any) -> TensorExpression | TensorNetwork:
```

Requires features: `python-stubs`

Exhaustive reference · Source

format_tensor

Format a tensor expression using compact Spenso notation.

Format a tensor expression using compact Spenso notation.

```
def format_tensor(expression: Expression, show_dimensions: Optional[bool] = None, *,
settings: Optional[DisplaySettings] = None) -> str:
```

Requires features: `python-stubs`

Exhaustive reference · Source

formatted

Build Symbolica's rich display wrapper for a tensor expression.

Build Symbolica's rich display wrapper for a tensor expression.

```
def formatted(expression: Expression, show_dimensions: Optional[bool] = None, *,
settings: Optional[DisplaySettings] = None, notation_source: Optional[str] = None) ->
FormattedOutput:
```

Requires features: `python-stubs`

Exhaustive reference · Source

set_symbolica_rayon_enabled

Configure whether Spenso may use Rayon for Symbolica operations.

Configure whether Spenso may use Rayon for Symbolica operations.

```
`Auto` checks the Symbolica license once during this call. When licensed,
operations may use Rayon and apply workload heuristics where available;
when unlicensed, symbolic operations remain serial. The returned boolean
reports whether the resolved policy permits Rayon, not whether every
operation will use it.
```

```
def set_symbolica_rayon_enabled(policy: SymbolicParallelism) -> bool:
```

Requires features: `python-stubs`

Exhaustive reference · Source

to_html

Render a tensor expression to semantic HTML through the optional Typst runtime.

Render a tensor expression to semantic HTML through the optional Typst runtime.

```
def to_html(expression: Expression, show_dimensions: Optional[bool] = None, *,
settings: Optional[DisplaySettings] = None, notation_source: Optional[str] = None) ->
str:
```

Requires features: `python-stubs`

Exhaustive reference · Source

`to_svg`

Render a tensor expression to an SVG string through the optional Typst runtime.

Render a tensor expression to an SVG string through the optional Typst runtime.

```
def to_svg(expression: Expression, show_dimensions: Optional[bool] = None, *,
settings: Optional[DisplaySettings] = None, notation_source: Optional[str] = None) ->
str:
```

Requires features: `python-stubs`

Exhaustive reference · Source

`to_typst`

Format a tensor expression as Typst math source.

Format a tensor expression as Typst math source.

```
def to_typst(expression: Expression, show_dimensions: Optional[bool] = None, *,
settings: Optional[DisplaySettings] = None) -> str:
```

Requires features: `python-stubs`

Exhaustive reference · Source

`trace`

Close an ordered factor sequence into a canonical cyclic trace.

Close an ordered factor sequence into a canonical cyclic trace.

```
def trace(representation: Representation, *factors: Any) -> TensorExpression |
TensorNetwork:
```

Requires features: `python-stubs`

Exhaustive reference · Source

Version information

Save these identifiers with published results and include them in issue reports so that collaborators can reproduce the same software environment.

- **Documentation channel:** `latest`
- **Documentation route:** `/products/spenso/latest/`
- **Source revision:** `8c92672b21772f91d479cb1f6c24b25b24a462de`

Package versions and enabled features

- `gammaloop/gammalooprs` — `0.3.4` (`no optional features`)
- `gammaloop/gammaloop-api` — `0.1.0` (`features: cli`)
- `gammaloop/gammaloop` — `0.3.4` (`features: python, abi3, python-stubs`)
- `linnet/linnet` — `0.17.0` (`features: native, serde, bincode, drawing, symbolica`)
- `linnet/linnet-py` — `0.1.0` (`features: extension-module, abi3`)
- `spenso/spenso` — `0.6.0` (`features: native, symbolica`)
- `spenso/spenso-macros` — `0.3.2` (`no optional features`)
- `spenso/spenso-hep-lib` — `0.1.7` (`features: native`)
- `spenso/spynso3` — `0.1.2` (`features: python-stubs`)
- `idenso/idenso` — `0.3.0` (`features: native, reference-cases`)
- `idenso/idenso` — `0.3.0` (`features: python, python-stubs`)
- `vakint/vakint` — `0.1.2` (`no optional features`)
- `vakint/vakint` — `0.1.2` (`features: python, python-stubs`)